



UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea in Ingegneria Informatica

Elaborato finale in **Fondamenti di misure**

***Reti Neurali Embedded per il
riconoscimento della Lingua
Italiana dei Segni***

Anno Accademico 2024/2025

Candidato
Giulia Coppola
matr. N46007104

Relatore
Ch.mo prof. Nicola Moccaldi
Ing. Francesco Caputo

*A chi vive nell'ombra dei conflitti,
a chi continua a cercare luce tra le macerie,
a chi parla anche quando il mondo non ascolta.*

*A chi ha imparato a restare in piedi da solo,
anche quando il mondo pesava troppo.*

*Alla mia famiglia,
che mi ha insegnato la forza dolce di chi ama senza condizioni.*

*Alla mia ombra che trema
e alla mia luce che non cede.*

Indice

Introduzione	1
1 Origini ed evoluzione del linguaggio dei segni	4
1.1 Ipotesi biologica e basi neuroscientifiche del linguaggio visivo	5
1.2 Storia e riconoscimento istituzionale delle lingue dei segni	7
1.3 Comunicazione e inclusione sociale	12
1.4 Interventi educativi e strumenti tecnologici a supporto	13
2 Tecnologie e approcci per il riconoscimento automatico della LIS	15
2.1 Il Duplice Processo di Interpretazione e i Paradigmi Metodologici	16
2.1.1 Sistemi basati su computer vision	17
2.1.2 Sistemi basati su dispositivi sensoriali	18
2.1.3 Sistemi Ibridi	19
2.2 Metodi di riconoscimento	20
2.2.1 Hand Landmarks e Machine Learning	20
2.2.2 Deep Learning	22

3	Ambienti embedded	25
3.1	Introduzione ai sistemi embedded e al paradigma TinyML	25
3.2	Ottimizzazione dei modelli di deep learning per ambienti a risorse limitate	28
3.2.1	Quantizzazione	28
3.2.2	Pruning	29
3.2.3	Knowledge distillation	30
3.2.4	Architetture light	31
3.3	Microcontrollore STM32N6	31
4	Dataset LIS-Detection: struttura, analisi e preprocessing	36
4.1	Origine e caratteristiche dei dati	36
4.2	Struttura: landmark e feature vettoriali	38
4.3	Analisi statistica e distribuzione delle classi	41
4.4	Preprocessing e suddivisione dei dati	43
5	Modellazione e training del classificatore neurale	45
5.1	Specifiche del modello e criteri di progettazione	45
5.2	Architettura scelta e training	46
5.3	Metriche di valutazione: accuratezza e matrice di confusione	50
6	Implementazione hardware e validazione	54
6.1	Architettura hardware e vincoli computazionali	55
6.2	Conversione del modello in TensorFlow Lite	57

6.3	Integrazione con STM32Cube.AI e inferenza sul dispositivo	59
6.3.1	Analisi dell'allocazione della memoria	59
6.3.2	Distribuzione delle epoche tra hardware e software	60
7	Implementazione software	63
7.1	Struttura generale e obiettivi funzionali	64
7.2	La funzione <code>perform_prediction()</code>	65
7.2.1	Allocazione delle risorse e preparazione dell'input	68
7.2.2	Serializzazione e invio del vettore di input . . .	69
7.2.3	Ricezione e decodifica della risposta	70
7.2.4	Inferenza locale per la validazione	71
7.3	Firmware e gestione dell'inferenza su STM32N6	72
7.3.1	Gestione della comunicazione UART	73
7.3.2	Ciclo operativo e parsing dell'input	75
7.3.3	Esecuzione dell'inferenza neurale	76
7.3.4	Trasmissione del risultato	78
	Conclusioni	80
	Bibliografia	84

Introduzione

Negli ultimi anni, la crescente attenzione verso l'inclusione e l'accessibilità ha portato la comunità scientifica e tecnologica a interrogarsi su come le innovazioni digitali possano contribuire a ridurre le barriere comunicative. Tra queste, una delle più significative riguarda il dialogo tra persone sorde e udenti, un ambito in cui la *lingua dei segni* riveste un ruolo fondamentale.

Le lingue dei segni, presenti in quasi tutti i Paesi del mondo, costituiscono veri e propri sistemi linguistici naturali, dotati di grammatica, sintassi e lessico propri, indipendenti dalle lingue vocali. Esse non sono semplici trasposizioni gestuali del linguaggio parlato, ma forme complete e autonome di espressione nate e sviluppate all'interno delle comunità sorde per soddisfare le loro esigenze comunicative e culturali.

La lingua dei segni, dunque, non rappresenta soltanto uno strumento di comunicazione, ma un patrimonio culturale e identitario capace di esprimere la complessità del pensiero umano attraverso la modalità visivo-gestuale. In Italia, tale patrimonio trova la sua espressione nella *Lingua dei Segni Italiana (LIS)*, che negli ultimi anni ha ricevuto

crescente riconoscimento sia a livello istituzionale che accademico.

Nonostante i progressi normativi che ne hanno sancito il riconoscimento ufficiale, la piena integrazione della lingua dei segni nella vita quotidiana incontra ancora numerosi ostacoli, dovuti soprattutto alla scarsità di interpreti, alla limitata diffusione del suo insegnamento e alla mancanza di strumenti tecnologici realmente efficaci per la traduzione automatica dei segni. In questo contesto, l'applicazione dell'*intelligenza artificiale* e della *computer vision* offre prospettive di grande interesse, aprendo la possibilità di trasformare i gesti in linguaggio scritto o parlato in modo immediato e naturale.

Il presente lavoro di tesi si inserisce in questo scenario, ponendosi come obiettivo la realizzazione di un sistema di riconoscimento automatico della *Lingua dei Segni Italiana* basato su tecniche di machine learning e deep learning, capace di identificare i gesti manuali e di convertirli in corrispondenti lettere dell'alfabeto. La ricerca si è concentrata in particolare sull'impiego di modelli ottimizzati per ambienti a risorse limitate, al fine di permetterne l'esecuzione diretta su microcontrollori embedded senza ricorrere a infrastrutture di calcolo esterne.

L'interesse di questa sperimentazione risiede non solo nell'aspetto tecnologico, ma anche nel suo potenziale impatto sociale: un sistema in grado di riconoscere la lingua dei segni in tempo reale, con dispositivi economici e portatili, può infatti costituire un ponte comunicativo

concreto tra comunità sorde e udenti, favorendo una società più equa e accessibile.

La tesi si propone dunque di esplorare la convergenza tra linguaggio, neuroscienze e intelligenza artificiale, mostrando come le tecnologie di riconoscimento visivo possano diventare strumenti di inclusione. L'obiettivo non è sostituire la competenza umana dell'interprete, ma ampliare le possibilità di interazione e contribuire alla costruzione di un ecosistema digitale capace di valorizzare la diversità linguistica e culturale.

Capitolo 1

Origini ed evoluzione del linguaggio dei segni

Le lingue dei segni non sono una semplificazione della lingua parlata, né un sistema secondario o mimico. Sono invece lingue naturali a pieno titolo, dotate di una propria struttura grammaticale, sintattica e morfologica, basata sulla modalità visivo-gestuale. L'uso delle mani, del volto e dello spazio tridimensionale consente una comunicazione estremamente ricca e articolata, capace di esprimere concetti astratti e sfumature semantiche al pari delle lingue vocali. Le lingue dei segni si sono sviluppate spontaneamente in molte comunità sorde nel corso della storia, in risposta al bisogno umano di comunicare. A differenza di quanto comunemente si crede, non esiste una lingua dei segni universale. Nel mondo sono riconosciute circa 130 lingue dei segni, ognuna dotata di proprie regole grammaticali, varianti regionali

e una storia distinta [1, 2]. Tra le più studiate vi sono l'American Sign Language (ASL), la British Sign Language (BSL), la Langue des Signes Française (LSF) e la Lingua dei Segni Italiana (LIS) [3]. Come approfondito nel paragrafo 1.2, nel corso del tempo le comunità sorde hanno sviluppato forme autonome di comunicazione visivo-gestuale, anche in contesti isolati o socialmente marginali. Alcuni ordini monastici medievali, ad esempio, elaborarono codici gestuali per rispettare i voti di silenzio, dimostrando che il linguaggio gestuale poteva essere un mezzo efficace anche in comunità udenti. La trasmissione intergenerazionale della lingua dei segni è stata spesso ostacolata da politiche educative e culturali fondate sull'oralismo ma, nonostante ciò, le lingue dei segni hanno continuato a evolversi e a sopravvivere, costituendo il fondamento dell'identità culturale sorda [4].

1.1 Ipotesi biologica e basi neuroscientifiche del linguaggio visivo

Dal punto di vista evolutivo, diversi studiosi sostengono l'ipotesi secondo la quale la comunicazione gestuale avrebbe preceduto quella orale nella storia dell'evoluzione umana (*gesture-first*). L'esistenza di lingue dei segni naturali, nate spontaneamente in comunità sorde, conferma che la modalità visuo-gestuale è perfettamente adatta all'elaborazione di contenuti linguistici complessi [5]. Le neuroscienze rafforzano

questa tesi: l'elaborazione della lingua dei segni attiva le stesse aree corticali deputate al linguaggio parlato, in particolare l'area di Broca (lobo frontale sinistro) e l'area di Wernicke (lobo temporale posteriore) Fig. 1.1. Tuttavia, nel caso delle lingue visuo-gestuali si osserva una maggiore attivazione delle aree occipito-parietali coinvolte nell'elaborazione spaziale e motoria [6]. Questo dimostra che il cervello umano è predisposto a rappresentare il linguaggio anche attraverso modalità visive.

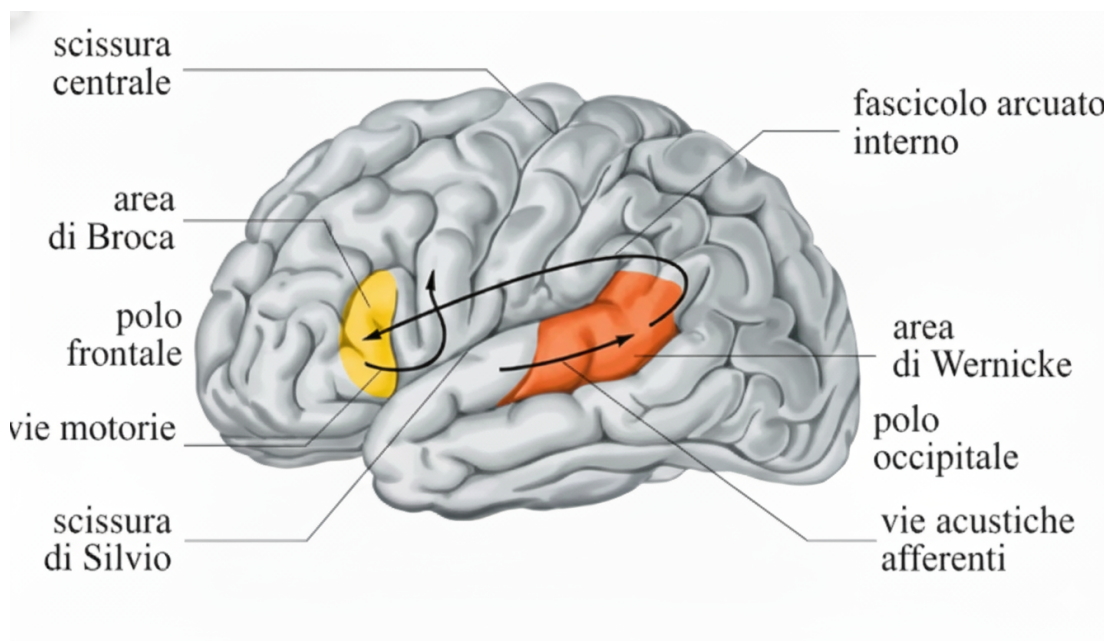


Figura 1.1: Modello di organizzazione del linguaggio proposto da Wernicke

Inoltre, i neuroni specchio, attivi sia nell'osservazione che nell'esecuzione di un gesto, sono stati identificati come possibili fondamenti neurobiologici della comprensione gestuale, fornendo un ponte tra percezione e produzione linguistica [7]. Dal punto di vista ontogenetico, i bambini sordi esposti precocemente a una lingua dei segni, come

la LIS, mostrano le stesse fasi di acquisizione linguistica dei bambini udenti: dalla lallazione manuale alla costruzione di frasi complesse e allo sviluppo lessicale [8]. Questo conferma che la capacità linguistica è indipendente dal canale sensoriale e che il cervello umano è predisposto per l'elaborazione di strutture grammaticali, qualunque sia la modalità di input.

1.2 Storia e riconoscimento istituzionale delle lingue dei segni

Fin dall'antichità, l'esigenza di comunicare ha spinto le comunità sorde a sviluppare sistemi visuo-gestuali, molto prima di qualsiasi riconoscimento formale. Tuttavia, l'atteggiamento verso la sordità era spesso improntato al pregiudizio. Nell'antica Grecia, pensatori come Aristotele sostenevano che senza l'udito e la parola non fosse possibile acquisire conoscenza e ragione [9]. Questa errata associazione tra sordità e presunta incapacità intellettuale si propagò per secoli, influenzando il diritto romano che arrivò a escludere i sordi dalla vita civile. Nonostante ciò, alcune testimonianze, come quella di Sant'Agostino, riconoscevano la capacità dei sordi di comunicare pienamente con i segni.

Allo stesso modo, alcuni ordini monastici, per rispettare i voti di silenzio, avevano sviluppato un proprio codice gestuale, un "linguaggio

dei segni monastico", che dimostrava l'efficacia e la funzionalità di tale forma di espressione. In alcune comunità isolate, dove l'incidenza della sordità era più elevata, si svilupparono persino delle vere e proprie lingue dei segni locali, utilizzate non solo dai sordi, ma anche dagli udenti, a riprova della loro naturalità e del loro valore [10].

Il Rinascimento segnò l'inizio di un'apertura e di un interesse crescente verso l'educazione dei sordi, in particolare grazie alla rivalutazione dei testi classici che portò a una più corretta interpretazione del pensiero di Aristotele. In questo periodo emersero figure pionieristiche. Il primo insegnante per sordi documentato fu il monaco benedettino Pedro Ponce de León in Spagna nel XVI secolo (Figura 1.2a). Il suo metodo, che combinava un alfabeto manuale con la scrittura e l'oralismo, fu rivoluzionario per l'epoca.

L'interesse di famiglie aristocratiche con eredi sordi fu un motore fondamentale per la sperimentazione educativa, poiché mirava a garantire ai figli il diritto di ereditare e la piena responsabilità giuridica. Nel XVII e XVIII secolo, figure come Juan Pablo Bonet e Johann Conrad Amman continuarono a sperimentare nuove metodologie, gettando le basi per il lungo dibattito tra oralismo e manualismo che avrebbe caratterizzato i secoli successivi. Juan de Pablo Bonet fu in particolare colui che plagiò il metodo di Ponce de León e creò il primo vocabolario di una lingua dei segni, l'“*Abecedario Demonstrativo*”, che gettò le basi per gli alfabeti manuali moderni (Figura 1.2b) [11].

CAPITOLO 1. ORIGINI ED EVOLUZIONE DEL LINGUAGGIO DEI SEGNI



Figura 1.2: (a) Una raffigurazione di Pedro Ponce de León che insegna ai suoi studenti. (b) Immagine dell'”Abecedario Demonstrativo” di J. P. Bonet.

Nel XVIII secolo, l’abate Charles-Michel de l’Épée fondò a Parigi la prima scuola pubblica per sordi. Il suo metodo si basava sull’uso dei segni naturali, sistematizzati in un linguaggio didattico, i segni metodici (Figura 1.3). Mentre de l’Épée promuoveva l’approccio manuale, il suo contemporaneo Samuel Heinicke, in Germania, sosteneva il metodo puramente oralista, proibendo l’uso dei segni [12]. Questa divisione segnò l’inizio di una lunga e controversa battaglia metodologica.



Figura 1.3: Litografia a colori dell'abate de L'Epée, fondatore dell'educazione dei sordomuti in Francia, sopra l'alfabeto della lingua dei segni francese.

Negli Stati Uniti, il manualismo si affermò grazie a Thomas Hopkins Gallaudet e Laurent Clerc, che nel 1817 fondarono ad Hartford (Connecticut) la prima scuola permanente per sordi. Da lì nacque la *Lingua dei Segni Americana* (ASL). Tuttavia, questo periodo di fioritura fu bruscamente interrotto dal Congresso Internazionale degli Educatori di Sordi di Milano del 1880 (Figura 1.4). Durante questo evento, una maggioranza di educatori udenti votò a favore dell'oralismo puro, dichiarando l'insegnamento del parlato superiore al linguaggio dei segni e bandendolo dalle scuole. Le conseguenze furono drammatiche: le lingue dei segni furono soppresse per quasi un secolo, costringendo

i bambini sordi a imparare a parlare e a leggere le labbra, spesso con risultati frustranti [13].



Figura 1.4: Sala conferenze al Congresso di Milano del 1880 [13]

Nel 1960 il linguista americano William Stokoe pubblicò uno studio rivoluzionario in cui dimostrò che la ASL possedeva una propria grammatica e struttura fonologica [14]. Introdusse il concetto di *Che-rema*, l'unità più piccola in cui può essere scomposto un segno di una lingua dei segni, e trattò i segni come unità linguistiche comparabili ai fonemi, dimostrando in modo scientifico che possedeva tutte le caratteristiche grammaticali e sintattiche di una lingua naturale. Il suo lavoro fu cruciale e aprì la strada al riconoscimento delle lingue dei segni come vere e proprie lingue a tutti gli effetti.

In Italia, la storia della *Lingua dei Segni Italiana* (LIS) rispecchia quella di altre lingue dei segni europee, con periodi di clandestinità e un riconoscimento tardivo. La prima scuola per sordi fu fondata a Roma nel 1784 dall'abate Tommaso Silvestri, ispirato dai metodi di de l'Épée. Tuttavia, l'Italia, in quanto paese ospitante del Congresso di

Milano, fu particolarmente colpita dalle sue risoluzioni. Le scuole italiane adottarono prevalentemente l'oralismo, relegando la LIS all'uso informale e clandestino. Nonostante ciò, il contributo di figure come Antonio Magarotto, che fondò numerose scuole per sordi, fu cruciale per la comunità sorda italiana [15].

Il riconoscimento giuridico è arrivato solo nel 2021, con l'articolo 34-ter del Decreto Sostegni, che ha formalmente riconosciuto la LIS e la LIS tattile (LIST) [16].

1.3 Comunicazione e inclusione sociale

L'uso della LIS va ben oltre il puro scambio linguistico: costituisce un vero e proprio strumento di inclusione che abbassa barriere sociali, promuove dignità e partecipazione attiva. Uno studio condotto in Italia su 182 adulti sordi ha evidenziato che chi ha acquisito la LIS sin dall'infanzia manifesta livelli significativamente minori di ansia sociale rispetto a chi non la conosce, suggerendo che la padronanza della lingua contribuisce anche al benessere psicologico e alla gestione delle relazioni sociali [17].

In diversi contesti educativi, l'inserimento della LIS favorisce l'inclusione degli studenti sordi nelle classi ordinarie. Un caso studio in Giordania ha mostrato come l'offerta di corsi di lingua dei segni all'università incrementi il senso di partecipazione e supporti lo sviluppo dell'identità di studenti con disabilità uditiva [18]. L'inclusione lin-

guistica ha anche una dimensione politica e mediatica: l'uso dell'inset (riquadro con interprete LIS nei programmi televisivi) contribuisce a dare visibilità alla comunità sorda e a integrare la loro lingua nei flussi informativi pubblici. In un'analisi sul ruolo dell'inset LIS durante eventi ufficiali, è emerso come esso possa promuovere l'accesso all'informazione per i sordi, ma solo se supportato da policy, standard tecnici e validazione culturale [19]. Infine, comunità coese che valorizzano la LIS sperimentano una maggiore partecipazione attiva nelle decisioni sociali. Dove la LIS è più diffusa, gli interlocutori udenti sono più propensi ad apprendere, riducendo l'isolamento linguistico. In tali contesti, la lingua dei segni diventa ponte e non barriera, promuovendo una cultura condivisa del dialogo.

1.4 Interventi educativi e strumenti tecnologici a supporto

Il contesto educativo è forse l'ambito in cui l'accesso alla LIS produce i risultati più tangibili. L'introduzione della lingua dei segni nei percorsi scolastici, l'impiego di interpreti LIS e l'adozione di materiali didattici visivi hanno dimostrato di migliorare notevolmente la partecipazione e il rendimento degli studenti sordi [20].

Accanto a queste strategie tradizionali, l'evoluzione tecnologica sta aprendo nuove frontiere per l'inclusione. Dispositivi e applicazioni di-

gitali basati su intelligenza artificiale sono in grado di riconoscere e tradurre la LIS in tempo reale, abbattendo molte delle barriere che ancora oggi separano sordi e udenti. Questi strumenti non si limitano alla semplice trascrizione: diventano veri e propri "ponti comunicativi", capaci di rendere naturale l'interazione tra individui con codici linguistici differenti [21].

La possibilità di tradurre istantaneamente i gesti e le espressioni facciali della LIS in testo o voce rappresenta un passo decisivo verso una società più inclusiva. Sistemi di sottotitolazione automatica, app mobili, avatar 3D e visori indossabili, stanno trasformando radicalmente l'accessibilità in tempo reale [22, 23].

Tali tecnologie trovano applicazione non solo nella vita quotidiana, ma anche in ambiti cruciali come l'istruzione, la sanità e il lavoro. I bambini sordi, ad esempio, possono usufruire di trascrizioni istantanee durante le lezioni; negli ambienti professionali, le tecnologie di traduzione automatica consentono maggiore autonomia, riducendo la necessità di interpreti [24].

In conclusione, l'integrazione della LIS nei percorsi educativi e l'adozione di strumenti tecnologici rappresentano due facce della stessa medaglia: garantire pari opportunità comunicative, formative e sociali, riconoscendo nella LIS non solo una lingua, ma una dimensione culturale, educativa e tecnologica del tutto attuale.

Capitolo 2

Tecnologie e approcci per il riconoscimento automatico della LIS

L'interpretazione della lingua dei segni è intrinsecamente complessa a causa della sua natura visuale e multimodale. Essa si compone di molteplici elementi, tra cui gesti manuali, che definiscono l'alfabeto, i numeri e le parole statiche, segnali di bocca (*mouthings cues*) ed espressioni facciali, che veicolano significati, sentimenti e aspetti grammaticali. Inoltre, come approfondito nel capitolo 1, la lingua dei segni non è universale, ed ogni paese possiede la propria sintassi e semantica specifica.

2.1 Il Duplice Processo di Interpretazione e i Paradigmi Metodologici

I sistemi di interpretazione della lingua dei segni gestiscono due processi principali e inversi per facilitare la comunicazione tra persone udenti e non udenti [25]:

1. *Lingua dei Segni a Testo/Voce*: Questo processo consente alle persone udenti di comprendere le persone sorde o mute. Prevede la cattura di immagini o video sequenziati, la pre-elaborazione (segmentazione, filtraggio), l'estrazione e il tracciamento continuo di mani e volto, l'estrazione delle feature, la classificazione dei segni, la conversione dei segni in testo e, infine, la sintesi vocale del testo.
2. *Testo/Voce a Lingua dei Segni*: Questo processo è progettato per aiutare le persone sorde a comprendere ciò che viene detto. L'input vocale viene tradotto in testo, che subisce una pre-elaborazione (tokenizzazione, lemmatizzazione), e il testo pre-elaborato viene quindi convertito in segni corrispondenti, visualizzati tramite un avatar animato o immagini statiche.

Il riconoscimento automatico della lingua dei segni, conosciuto come *Sign Language Recognition* (SLR), si basa primariamente su tre approcci per l'acquisizione del gesto: sistemi basati su *computer vision*,

su *dispositivi sensoriali*, o su *configurazioni ibride*. Ciascun approccio presenta caratteristiche distinte in termini di accuratezza, scalabilità, robustezza e applicabilità in contesti reali.

2.1.1 Sistemi basati su computer vision

I sistemi basati su computer vision utilizzano flussi video acquisiti da telecamere RGB, stereoscopiche o a profondità per analizzare gesti e posture. Questi sistemi sono meno invasivi, più economici e maggiormente compatibili con dispositivi di largo consumo [26, 27].

L'elaborazione prevede una pipeline strutturata:

1. *Acquisizione video*: il segnante viene registrato tramite telecamere standard o di profondità.
2. *Preprocessing*: le immagini vengono normalizzate, ridimensionate e segmentate per migliorarne la qualità.
3. *Estrazione di caratteristiche*: strumenti come MediaPipe rilevano 21 landmark tridimensionali della mano in tempo reale, oltre a punti chiave di viso e corpo.
4. *Classificazione*: le sequenze di punti vengono analizzate da modelli di deep learning che associano i movimenti a specifici segni.

I limiti principali sono legati alla variabilità ambientale (luce, sfondo, occlusioni), alla qualità delle immagini [26, 27], e le occlusioni, quando una parte della mano o del corpo è nascosta.

Ciononostante, la portabilità e la naturalezza d'uso rendono questo approccio il più promettente per applicazioni in contesti reali.

2.1.2 Sistemi basati su dispositivi sensoriali

I sistemi di riconoscimento del linguaggio dei segni basati su dispositivi sensoriali utilizzano strumenti indossabili per acquisire dati biomeccanici, come la flessione delle dita e i movimenti del braccio. Questi sistemi offrono un'elevata precisione, ma presentano limiti di natura ergonomica ed economica che ne ostacolano la diffusione su larga scala.

Alcuni esempi di sistemi sensorizzati possono essere:

- *Guanti sensorizzati*: guanti che integrano sensori di flessione e movimento per catturare la postura e la dinamica della mano [28, 29]. I dati raccolti vengono trasmessi via Bluetooth a un'unità centrale di elaborazione, che li confronta con un database per generare una trascrizione testuale o audio del gesto. Nonostante l'accuratezza elevata nel rilevamento, l'invasività dell'hardware e la limitata naturalezza dell'interazione ne confinano l'uso a contesti di laboratorio.
- *Bracciali sensorizzati*: Bracciali [30, 31] che utilizzano sensori di elettromiografia di superficie (sEMG) combinati con unità di misura inerziale (IMU) per rilevare l'attività muscolare e il movimento del braccio. Questo approccio ha mostrato una precisione

del 96.16% nel riconoscimento di gesti ASL. Tuttavia, la necessità di contatto fisico e la calibrazione individuale restano elementi critici.

2.1.3 Sistemi Ibridi

I sistemi ibridi combinano tecniche visive e sensoriali per migliorare l'affidabilità del riconoscimento in condizioni complesse. Nonostante l'efficacia potenziale, le sfide tecniche ed economiche, come l'alto costo dell'hardware, la complessità computazionale e la difficoltà di calibrazione, ne limitano attualmente l'adozione su vasta scala.

A titolo comparativo, la seguente tabella riassume i principali vantaggi e svantaggi di ciascun approccio, evidenziando il motivo per cui la ricerca si sta spostando massicciamente verso i sistemi basati sulla visione.

Criteri	Approcci Basati su Dispositivi Sensoriali	Approcci Basati sulla Visione
Costo	Elevato, a causa dell'hardware specializzato	Basso, utilizzabile con telecamere standard
Precisione	Generalmente alta, grazie al tracciamento diretto dei sensori	Varia in base alle condizioni ambientali, ma in miglioramento
Naturalezza	Bassa, richiede hardware scomodo e invasivo	Alta, l'utente interagisce in modo naturale
Portabilità	Limitata dalla necessità di hardware specifico e calibrazione	Alta, può essere integrata in dispositivi mobili
Sfide Tecniche	Ingombro, costi, calibrazione complessa	Illuminazione, sfondi, occlusioni, variazione del colore della pelle

L'attuale direzione della ricerca, concentrata prevalentemente sui sistemi *vision-based*, suggerisce che l'obiettivo primario del settore è raggiungere la scalabilità e l'accessibilità attraverso l'utilizzo di dispositivi a basso costo e onnipresenti, come gli smartphone. Ciò spinge

verso l'adozione di architetture di Deep Learning (DL) ottimizzate e framework software specifici per la gestione efficiente dei dati video.

2.2 Metodi di riconoscimento

2.2.1 Hand Landmarks e Machine Learning

Hand Landmarks

Un primo filone di ricerca si concentra sull'uso dei *landmark*, ossia punti chiave della mano (polso, nocche, falangi, estremità delle dita) che ne descrivono in modo sintetico la configurazione. Strumenti come *MediaPipe Hands* hanno reso possibile, con buona affidabilità, l'estrazione in tempo reale di 21 landmark tridimensionali per mano (Figura 2.1) [32]. Una volta acquisite le coordinate, queste vengono di norma normalizzate a un punto per compensare variazioni di scala o traslazione e trasformate in vettori compatti che possono essere classificati da algoritmi leggeri.

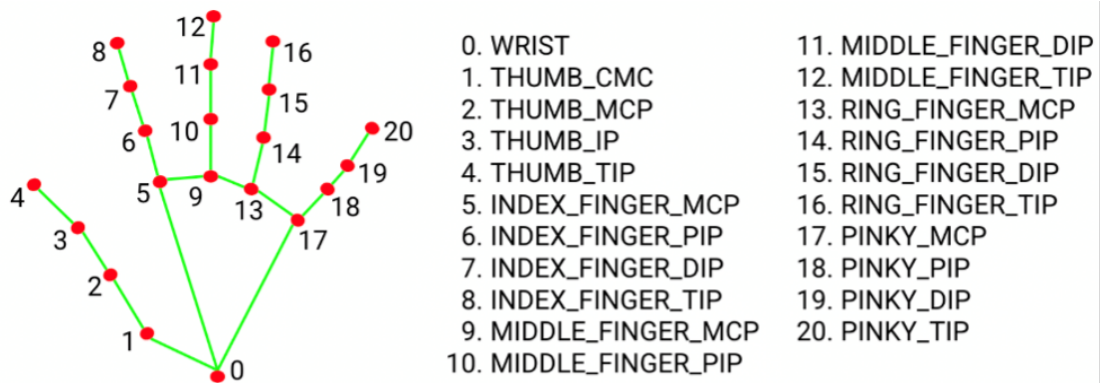


Figura 2.1: Anatomia del palmo della mano con 21 landmark [32].

Machine Learning

A partire dai landmark estratti dalle mani è possibile calcolare feature derivate che descrivono con maggiore precisione la configurazione e la dinamica del gesto.

Tra le più utilizzate si trovano:

- le distanze euclidee tra coppie di punti chiave, utili per catturare la geometria della mano;
- gli angoli articolari tra le falangi, che forniscono informazioni sull'apertura e sulla piegatura delle dita;
- le traiettorie temporali, che descrivono lo spostamento dei punti nello spazio nel corso del tempo.

Questi descrittori vengono in genere utilizzati come input per classificatori supervisionati quali le Support Vector Machines (SVM), i Decision Tree o i k-Nearest Neighbors (k-NN). Si tratta di modelli relativamente semplici da addestrare, che non richiedono tempi computazionali eccessivi né grandi quantità di dati.

Un ulteriore vantaggio riguarda la loro trasparenza: è possibile, ad esempio, risalire a quali feature (distanze, angoli, traiettorie) abbiano inciso maggiormente sulla decisione finale del modello, qualità particolarmente utile in contesti di ricerca e analisi. Questo spiega perché tali tecniche siano state a lungo preferite nelle prime fasi di sviluppo dei

sistemi di riconoscimento della lingua dei segni, soprattutto quando i dataset disponibili erano di dimensioni ridotte.

Il limite principale di questo approccio risiede però nella dipendenza dall'ingegnerizzazione manuale delle feature. Le caratteristiche devono essere definite dall'esperto in fase di progettazione e questo comporta che il modello apprenda solo ciò che è stato esplicitamente descritto. In altre parole, se un gesto presenta variazioni non previste – come inclinazioni della mano diverse, esecuzioni più rapide o movimenti leggermente sovrapposti – il classificatore rischia di non riconoscerlo correttamente. Ciò riduce la flessibilità e la capacità di generalizzazione, rendendo questi sistemi adatti soprattutto a scenari controllati (ad esempio in laboratorio), ma meno efficaci in contesti reali, dove la lingua dei segni è caratterizzata da una naturale variabilità espressiva, da condizioni ambientali mutevoli e da gesti complessi che sfuggono a una descrizione rigida tramite poche feature geometriche.

2.2.2 Deep Learning

Negli ultimi anni, l'attenzione della ricerca si è spostata in modo quasi unanime verso il *deep learning*, che ha segnato un netto salto di qualità nel riconoscimento automatico della LIS. A differenza degli approcci tradizionali, le reti profonde sono in grado di apprendere autonomamente le rappresentazioni più rilevanti per distinguere i gesti, eliminando la necessità di una progettazione manuale delle feature.

Le architetture più diffuse includono:

- *Convolutional Neural Networks* (CNN): particolarmente efficaci per i segni statici, in cui la configurazione della mano rimane invariata
- *Recurrent Neural Networks* (RNN) e *Long Short-Term Memory* (LSTM): sono capaci di modellare la sequenzialità temporale dei gesti dinamici
- *3D Convolutional Neural Networks* (3D-CNN): trattano intere sequenze video come volumi spazio-temporali, catturando simultaneamente informazioni di forma e movimento
- *Transformer*: grazie al meccanismo di self-attention riescono a individuare relazioni a lungo raggio nei dati sequenziali, aprendo la strada a sistemi in grado di affrontare il riconoscimento continuo della lingua dei segni

L'adozione di queste architetture ha permesso di ottenere accuratezza e capacità di generalizzazione senza precedenti, rendendo il deep learning lo standard di riferimento per il riconoscimento dei gesti.

Allo stesso tempo, però, esse presentano un costo computazionale molto elevato, dato che la complessità dei modelli li rende difficilmente compatibili con l'esecuzione in tempo reale su dispositivi mobili o microcontrollori.

Per questo motivo, una parte consistente della ricerca più recente si concentra sull'ottimizzazione e alleggerimento dei modelli, con l'obiettivo di coniugare elevate prestazioni con vincoli stringenti di memoria, potenza di calcolo e consumo energetico.

Queste strategie, che saranno approfondite nel paragrafo successivo, rappresentano la chiave per rendere realmente fruibili le tecniche di deep learning in applicazioni embedded e in scenari di utilizzo quotidiano.

Capitolo 3

Ambienti embedded

3.1 Introduzione ai sistemi embedded e al paradigma TinyML

Negli ultimi anni, la convergenza tra intelligenza artificiale e dispositivi embedded ha dato origine a un nuovo paradigma tecnologico noto come Tiny Machine Learning (TinyML), dove i modelli di machine learning e deep learning sono eseguiti direttamente su microcontrollori o sistemi a basso consumo energetico [33, 34].

Questo approccio consente di portare l’elaborazione “intelligente” vicino alla sorgente dei dati, eliminando la necessità di connessioni costanti al cloud e riducendo drasticamente latenza, costi e dipendenza dalla rete.

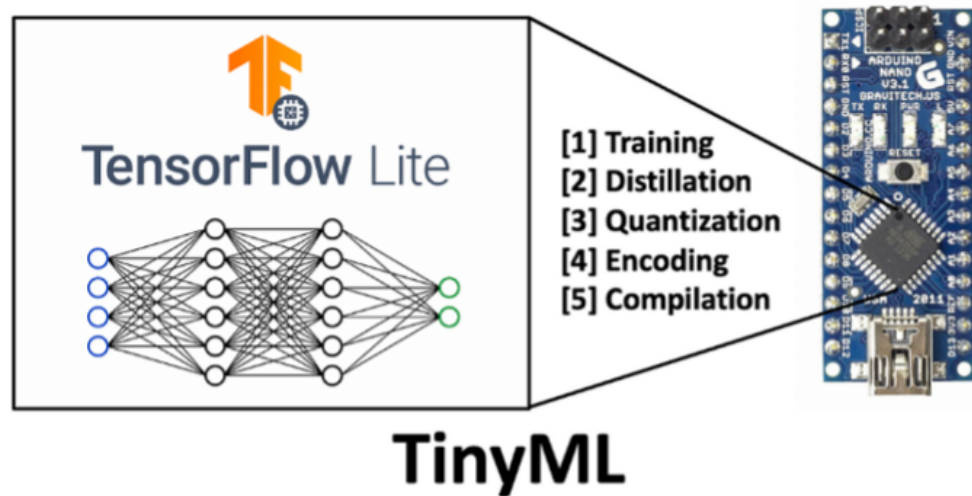


Figura 3.1: Microcontrollore che implementa TinyML con TensorFlow Lite

Come mostrato in Figura 3.1, il TinyML consente di eseguire modelli di apprendimento automatico direttamente su dispositivi a risorse limitate, come microcontrollori, grazie a framework ottimizzati come TensorFlow Lite.

Tuttavia, l'implementazione di reti neurali in contesti embedded rappresenta una delle sfide ingegneristiche più complesse del momento. A differenza di GPU e server ad alte prestazioni, i microcontrollori dispongono di risorse fortemente limitate:

- *Limitata capacità di memoria:* i modelli devono risiedere interamente nella memoria Flash e utilizzare quantità minime di RAM.
- *Vincoli energetici:* i dispositivi portatili devono garantire lunga

autonomia, imponendo un budget energetico ridotto per ogni inferenza.

- *Latenza ridotta*: il riconoscimento dei gesti deve avvenire in breve tempo (<100 ms) per mantenere un'interazione naturale.
- *Assenza di acceleratori dedicati*: molti MCU non dispongono di GPU o NPU, richiedendo ottimizzazioni specifiche per CPU general-purpose.

In applicazioni come il riconoscimento della lingua dei segni, queste limitazioni diventano ancora più critiche, dal momento che il sistema deve elaborare sequenze di landmark o immagini a frame rate costante, e fornire una classificazione accurata in tempo reale.

Ciò richiede modelli ultraleggeri, progettati per ottimizzare la memoria e massimizzare la velocità d'inferenza, senza degradare significativamente la precisione del riconoscimento.

Per affrontare tali sfide, la comunità scientifica e industriale ha sviluppato tecniche di compressione e ottimizzazione dei modelli neurali che riducono dimensione, complessità e consumo energetico mantenendo prestazioni accettabili.

3.2 Ottimizzazione dei modelli di deep learning per ambienti a risorse limitate

L'ottimizzazione dei modelli di deep learning per ambienti embedded mira a ridurre il numero di parametri e la complessità computazionale delle operazioni, mantenendo la capacità di generalizzazione del modello.

Le tecniche più efficaci si articolano in cinque categorie principali: quantizzazione, pruning, knowledge distillation, architetture light e framework di deployment ottimizzati.

3.2.1 Quantizzazione

La quantizzazione rappresenta la tecnica più diffusa e consolidata per il deployment di modelli in ambienti embedded. Essa consiste nella conversione della rappresentazione numerica dei pesi e delle attivazioni del modello da precisione a 32 bit in virgola mobile (FP32) a formati a precisione ridotta, tipicamente INT8, INT4 o anche binary quantization [35, 36].

I principali vantaggi della quantizzazione sono:

- riduzione del footprint di memoria fino al 75–90%;
- accelerazione delle operazioni aritmetiche grazie all'elaborazione intera nativa supportata da molte MCU;

- diminuzione del consumo energetico, con risparmi fino al 60% per inferenza [37].

Studi recenti dimostrano che l'uso della quantizzazione INT8 può mantenere una perdita di accuratezza inferiore all'1% rispetto al modello FP32, a fronte di un miglioramento fino a 3,5x nei tempi di inferenza su MCU STM32 e Jetson Nano [38, 33].

Tuttavia, modelli sensibili alla scala dei valori, come reti ricorrenti o convoluzionali temporali (TCN), possono subire un degrado più marcato. Per tali casi, si impiega la Quantization-Aware Training (QAT), che simula l'effetto della quantizzazione durante l'addestramento, migliorando la robustezza numerica [39].

3.2.2 Pruning

Il pruning consiste nell'eliminazione dei pesi o dei filtri che contribuiscono marginalmente all'output della rete, riducendo così la complessità del modello.

Si distinguono due approcci principali:

- *Pruning non strutturato*: rimuove singoli pesi e genera matrici sparse (efficace solo su hardware che supporta operazioni sparse).
- *Pruning strutturato*: elimina interi canali, filtri o layer, producendo modelli più compatti e accelerabili su MCU convenzionali [40, 41].

Il pruning strutturato può ridurre fino al 50% delle operazioni MAC e al 40% la memoria richiesta, con perdita di accuratezza contenuta entro l'1% [42].

Tuttavia, un pruning eccessivo può introdurre squilibri nei layer e peggiorare la stabilità numerica. È pertanto raccomandato applicare strategie iterative e valutare le metriche di accuratezza a ogni ciclo di riduzione.

3.2.3 Knowledge distillation

La *knowledge distillation* prevede l'uso di un modello complesso, chiamato *teacher*, che trasferisce le proprie conoscenze a un modello più leggero, detto *student*, permettendo di ridurre i costi computazionali senza compromettere drasticamente la precisione. Durante il processo, lo *student* non impara solo dai dati etichettati, ma anche dalle “soft label” (distribuzioni di probabilità) generate dal *teacher*, che contengono informazioni più ricche rispetto alle tradizionali etichette one-hot. Questo consente al modello *student* di apprendere relazioni fra classi, strutture interne e bias del modello *teacher*. In ambito embedded o TinyML, la distillazione è particolarmente utile: combinata con quantizzazione e pruning, permette di ottenere modelli ultra-leggeri adatti a MCU e dispositivi a risorse limitate, mantenendo un'accuratezza competitiva. Inoltre, alcune varianti includono la distillazione di caratteristiche intermedie (feature-based), relazioni fra esempi (relation-

based) o l'uso di più *teacher* (multi-teacher distillation) per migliorare ulteriormente le prestazioni del modello *student* [43].

3.2.4 Architetture light

Parallelamente alle tecniche di compressione, l'impiego di architetture progettate nativamente per dispositivi mobili rappresenta una soluzione complementare.

Reti come MobileNetV3, EfficientNet-Lite, SqueezeNet e ShuffleNet utilizzano strategie di progettazione “design-aware” che riducono il numero di operazioni MAC (Multiply–Accumulate) e la complessità dei layer convoluzionali [44, 45].

Ad esempio, MobileNetV3 sfrutta convoluzioni separabili in profondità (Depthwise Separable Convolution) e blocchi residui invertiti, raggiungendo accuratezze comparabili a reti molto più grandi, ma con un numero di parametri ridotto dell'80% [45]. Tali modelli sono particolarmente adatti ai microcontrollori, in quanto riducono l'uso di memoria intermedia e semplificano la mappatura delle operazioni sull'hardware.

3.3 Microcontrollore STM32N6

La famiglia di microcontrollori STM32N6 di STMicroelectronics rappresenta una delle piattaforme più avanzate nel panorama dell'intelli-

genza artificiale integrata (*embedded AI*), progettata per combinare la flessibilità dei microcontrollori tradizionali con la capacità di calcolo necessaria all'esecuzione di modelli di *deep learning* in tempo reale. Essa incarna il paradigma dell'*edge computing*, spostando l'elaborazione dell'intelligenza artificiale direttamente sul dispositivo, senza la necessità di connessione a server o infrastrutture cloud [46].

Dal punto di vista architetturale, il microcontrollore STM32N6 si basa su un core *ARM Cortex-M55*, una CPU ottimizzata per prestazioni elevate e basso consumo, dotata di estensioni SIMD Helium che accelerano le operazioni vettoriali e di *digital signal processing* [47].

L'elemento che maggiormente differenzia questa serie è la presenza di un'acceleratore neurale integrato, denominato *Neural Architecture and Runtime Technology (Neural-ARTTM)*, un'unità hardware dedicata all'esecuzione di reti neurali quantizzate in formato INT8 [46].

Quest'unità è specificamente progettata per gestire in modo estremamente efficiente le operazioni convoluzionali e matriciali che costituiscono il nucleo computazionale delle moderne architetture di deep learning.

L'acceleratore neurale lavora in stretta sinergia con il processore principale: mentre la CPU gestisce la logica di controllo, la pianificazione dei task e l'interfaccia con i sensori, il *Neural-ARTTM* esegue le operazioni intensive, riducendo significativamente la latenza e il consumo energetico durante l'inferenza.

L'interconnessione tra i due sottosistemi è gestita da un bus multilivello (*AXI*), che permette un rapido trasferimento dei dati, e da blocchi di *Tightly Coupled Memory (TCM)* ad accesso veloce, collocati in prossimità dell'acceleratore per minimizzare i tempi di accesso e i colli di bottiglia [48].

In Figura 3.2, è riportata l'architettura del microcontrollore STM32N6 in cui si nota il sottosistema ARM Cortex-M connesso all'acceleratore neurale dedicato (Neural-ART™) tramite il bus multilivello per la gestione efficiente dello scambio dati e l'accesso alla memoria.

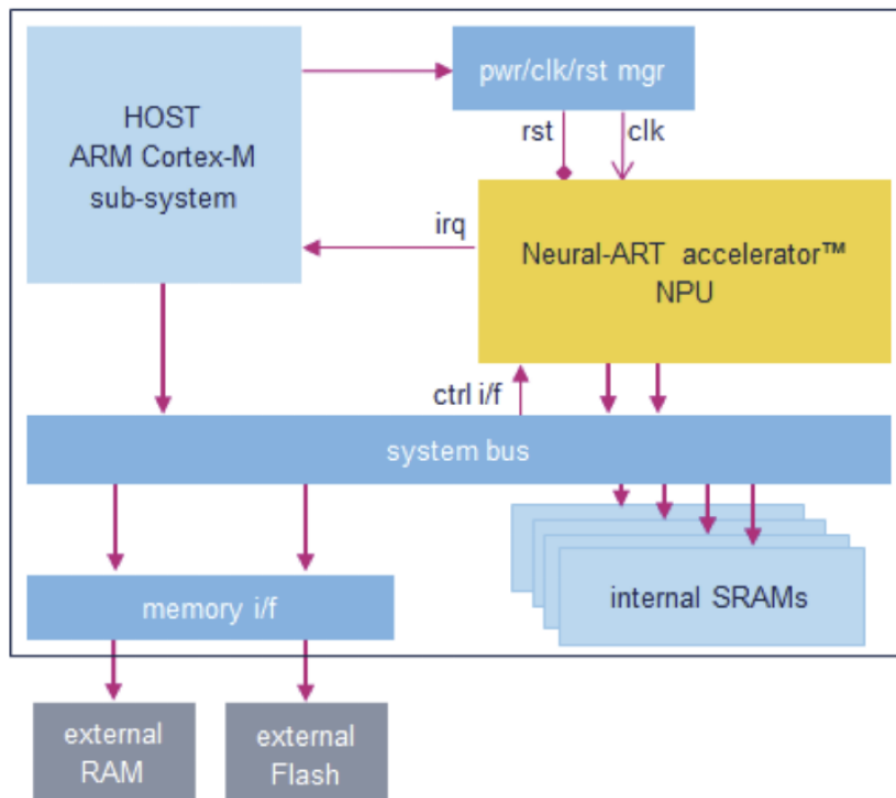


Figura 3.2: Architettura del microcontrollore STM32N6 con acceleratore neurale Neural-ART™.

In termini di prestazioni, il microcontrollore STM32N6 è in grado di raggiungere centinaia di giga-operazioni al secondo (GOPS) con un consumo di potenza elettrica inferiore ai 200 mW, offrendo un'efficienza energetica dell'ordine di 1 TFLOPS/W (teraflops per watt) [46].

Tali caratteristiche rendono questo dispositivo particolarmente adatto ad applicazioni di computer vision o gesture recognition in tempo reale, come il riconoscimento della lingua dei segni, dove è richiesta un'elaborazione locale a bassa latenza e a ridotto consumo.

Parallelamente all'hardware, STMicroelectronics fornisce un ecosistema software completo che facilita l'implementazione di modelli di deep learning. Lo strumento principale è *STM32Cube.AI*, una piattaforma che consente di importare modelli pre-addestrati sviluppati con *TensorFlow*, *PyTorch* o *ONNX*, convertirli automaticamente nel formato compatibile con la MCU e ottimizzarli per l'esecuzione sul Neural-ART™ [49]. Durante il processo di conversione, il tool analizza la topologia della rete, assegna automaticamente le operazioni più onerose all'acceleratore neurale e genera codice C altamente ottimizzato per la compilazione ed esecuzione diretta sulla scheda.

STM32Cube.AI include inoltre un profilatore integrato in grado di stimare in anticipo il tempo di inferenza, l'occupazione di memoria e il consumo energetico, fornendo così un'analisi predittiva delle prestazioni prima del deployment effettivo [49].

Grazie a questa sinergia tra hardware specializzato e strumenti

software, la famiglia di microcontrollori STM32N6 consente di eseguire modelli quantizzati con latenze inferiori ai 10 ms e con una riduzione dei consumi fino al 70% rispetto a implementazioni puramente software su CPU [34]. Queste prestazioni, unite alla compatibilità con framework di sviluppo diffusi, rendono la piattaforma particolarmente indicata per la realizzazione di sistemi autonomi e indossabili, capaci di elaborare i segnali sensoriali e riconoscere i movimenti della mano in tempo reale, senza dipendere da risorse esterne.

Capitolo 4

Dataset LIS-Detection: struttura, analisi e preprocessing

4.1 Origine e caratteristiche dei dati

Il dataset utilizzato per l'addestramento e la valutazione del sistema di riconoscimento della Lingua dei Segni Italiana (LIS) deriva dal progetto open source *LIS-Detection* [50].

Questo dataset è stato realizzato specificamente per il riconoscimento automatico delle lettere dell'alfabeto manuale LIS, con l'obiettivo di fornire un insieme di dati accessibile e adatto all'addestramento di modelli di *machine learning* e *deep learning*.

I dati sono stati raccolti tramite uno strumento di acquisizione dedicato, denominato *acqTool*, che utilizza la libreria *OpenCV* per la gestione della videocamera e il framework **MediaPipe Hands** di Google per l'individuazione dei punti di riferimento (landmark) della mano. Durante l'acquisizione, ogni gesto corrispondente a una lettera dell'alfabeto LIS è stato ripreso dalla telecamera e convertito in una rappresentazione numerica composta dalle coordinate bidimensionali dei principali punti anatomici della mano (polso, articolazioni, punte delle dita). L'insieme dei dati ottenuti è stato successivamente pubblicato anche sulla piattaforma *Kaggle* con il titolo "Right Hand Markers Dataset", al fine di favorire la condivisione e la riproducibilità della ricerca.

Il dataset comprende *26 classi*, corrispondenti alle lettere dalla *A* alla *Z*, con *500 campioni per classe*, per un totale di *13.000 istanze*. Ogni campione è rappresentato da *42 valori numerici*, corrispondenti alle coordinate x e y dei 21 landmark della mano rilevati da MediaPipe. Le coordinate sono state normalizzate nell'intervallo $[0, 1]$ mediante uno *scaler* Min-Max, in modo da eliminare le dipendenze dalle dimensioni in pixel o dalla distanza della mano rispetto alla telecamera.

La raccolta è stata condotta utilizzando esclusivamente la *mano destra*, scelta che semplifica il modello di classificazione evitando le variazioni dovute alla lateralità.

Nella Tabella 4.1 vengono riassunte le principali caratteristiche del

dataset LIS-Detection.

Caratteristica	Descrizione
Classi	26 (lettere dell'alfabeto LIS, da <i>A</i> a <i>Z</i>)
Campioni totali	13.000 (500 esempi per ciascuna classe)
Feature per campione	42 coordinate (21 punti x, y della mano)
Intervallo di valori	$[0,1]$ per ogni coordinata (dati normalizzati Min-Max)
Dominanza manuale	Solo configurazioni di mano destra acquisite

Tabella 4.1: Principali caratteristiche del dataset LIS-Detection.

Queste caratteristiche delineano un dataset bilanciato e specifico per il compito di classificazione di lettere LIS. L'impiego dei landmark come forma di rappresentazione, invece delle immagini grezze, consente una descrizione compatta ed efficace della postura della mano, riducendo la complessità computazionale e migliorando la velocità di elaborazione, aspetto fondamentale per applicazioni embedded e in tempo reale.

4.2 Struttura: landmark e feature vettoriali

Ogni istanza del dataset è descritta da un vettore di 42 feature, corrispondenti alle coordinate normalizzate (x, y) dei 21 landmark della mano individuati dal modello *MediaPipe Hands*. Questi punti rappresentano le articolazioni principali e le estremità delle dita, come illustrato in Figura 4.1, e definiscono in modo univoco la configurazione geometrica della mano.

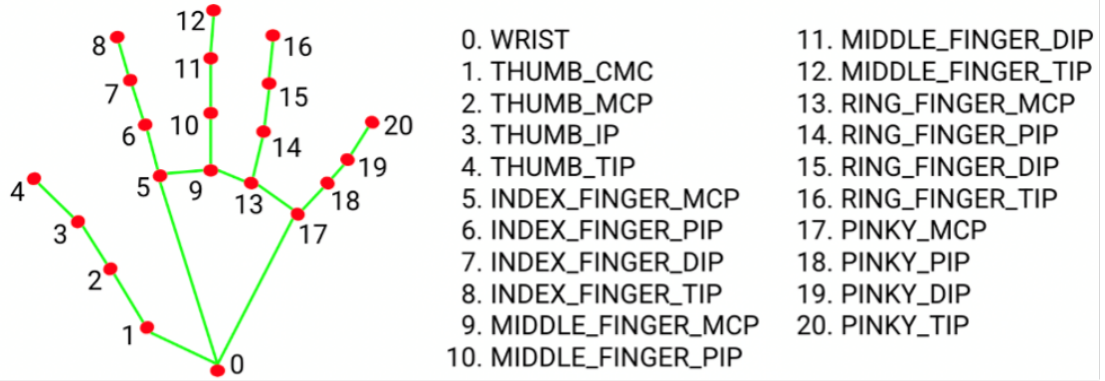


Figura 4.1: Schema dei 21 landmark della mano definiti dal modello *MediaPipe Hands*

La numerazione dei landmark segue la convenzione adottata da MediaPipe: il punto 0 identifica il polso, mentre i successivi definiscono le articolazioni metacarpali, interfalangee e le punte delle dita. Ad esempio, i punti 1–4 rappresentano il pollice (dalla base alla punta), 5–8 l’indice, 9–12 il medio, 13–16 l’anulare e 17–20 il mignolo.

Ciascun campione è quindi formalmente rappresentato dal vettore Eq.4.1

$$V_i = (x_1, y_1, x_2, y_2, \dots, x_{21}, y_{21}) \quad (4.1)$$

Le coordinate x e y sono comprese tra 0 e 1, dove 0 indica il bordo sinistro o superiore dell’inquadratura e 1 il bordo destro o inferiore. Questo tipo di rappresentazione rende i campioni *invarianti rispetto alla scala e alla posizione*, permettendo di confrontare gesti provenienti da diverse acquisizioni o soggetti.

La struttura dei file segue una logica gerarchica: all’interno della

directory principale del dataset, ogni sottocartella è dedicata a una lettera e contiene due file binari, denominati *vectors.pkl* e *classes.pkl*. Il primo contiene le matrici dei vettori di feature, mentre il secondo memorizza le etichette corrispondenti.

Durante la fase di caricamento, i file vengono letti, combinati e aggregati per formare due array principali:

- una matrice $X \in R^{13.000 \times 42}$, che raccoglie tutti i vettori di feature (42 coordinate normalizzate per ciascuno dei 13.000 campioni).
- un vettore $y \in \{0, 1, \dots, 25\}^{13.000}$, che contiene le etichette numeriche corrispondenti alle lettere A–Z.

Le etichette, originariamente in formato testuale, vengono convertite in valori numerici mediante la classe `LabelEncoder` di *scikit-learn*, consentendo un’elaborazione più efficiente durante le fasi di addestramento e validazione del modello.

Tale struttura garantisce un accesso sistematico ai dati, semplificando sia la fase di analisi statistica sia le successive elaborazioni di *machine learning*.

Per maggiore chiarezza, la Figura 4.2 mostra un estratto del codice Python utilizzato per la lettura e l’unione dei dati all’interno del file *train_model.py*, in cui viene effettuata la scansione ricorsiva delle directory e il caricamento dei file *vectors.pkl* e *classes.pkl*.

```
for label_folder in sorted(os.listdir(DATASET_DIR)):
    folder_path = os.path.join(DATASET_DIR, label_folder)
    for root, _, files in os.walk(folder_path):
        vector_file = next((os.path.join(root, f) for f in files if f.startswith("vectors")), None)
        class_file = next((os.path.join(root, f) for f in files if f.startswith("classes")), None)
        if vector_file and class_file:
            with open(vector_file, "rb") as vf, open(class_file, "rb") as cf:
                vectors = pickle.load(vf)
                classes = pickle.load(cf)
                for v, c in zip(vectors, classes):
                    if len(v) == 42: # usa solo quelli con 42 feature
                        X.append(v)
                        y.append(c)
```

Figura 4.2: Estratto di codice per il caricamento dei dati del dataset LIS-Detection

4.3 Analisi statistica e distribuzione delle classi

L'analisi della distribuzione delle classi conferma che il dataset LIS-Detection è perfettamente bilanciato: ciascuna delle 26 lettere dell'alfabeto è rappresentata da esattamente 500 campioni (Fig. 4.3). Questo equilibrio è fondamentale per l'addestramento di modelli di classificazione, poiché evita fenomeni di class *imbalance*, nei quali alcune classi risulterebbero sovra- o sotto-rappresentate, influenzando negativamente le prestazioni.

La numerosità complessiva di 13.000 campioni risulta adeguata rispetto alla dimensionalità ridotta delle 42 feature. Ogni classe dispone di un numero sufficiente di esempi per catturare le piccole variazioni con cui la stessa lettera può essere realizzata da soggetti diversi o in condizioni di ripresa non identiche. Tale variabilità intra-classe contri-

buisce ad aumentare la capacità del modello di generalizzare a nuove istanze.

La scelta di rappresentare ogni gesto tramite coordinate normalizzate e non tramite immagini complete riduce inoltre la varianza introdotta da fattori esterni, come l'illuminazione o lo sfondo.

Il dataset risulta dunque pulito, omogeneo e ben strutturato, caratteristiche che ne fanno una base adatta per addestrare e valutare modelli di apprendimento automatico mirati al riconoscimento gestuale.

Le analisi quantitative e grafiche più approfondite, come la distribuzione per classe e le correlazioni tra feature, verranno presentate nei capitoli successivi, in relazione ai risultati di addestramento.

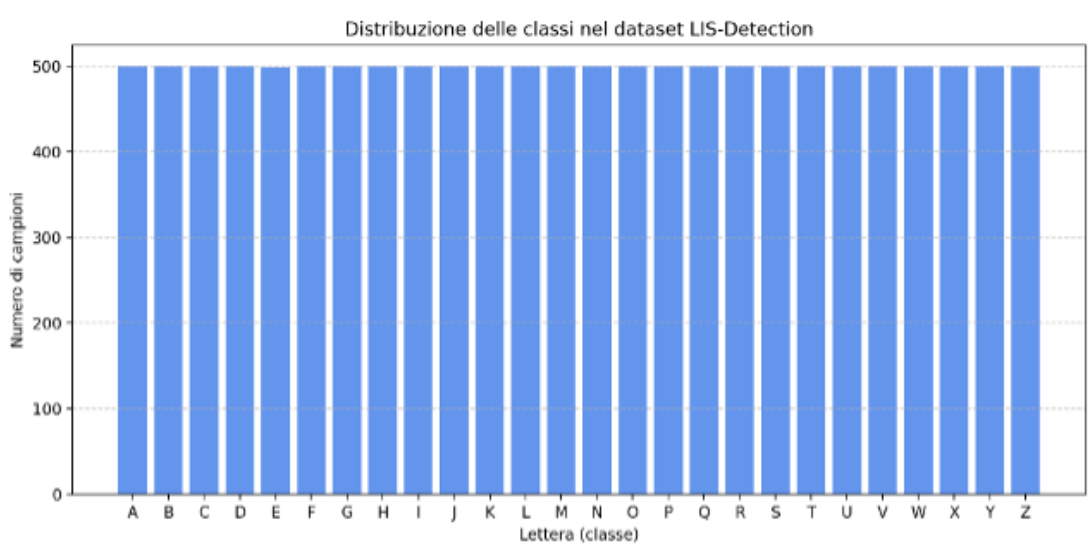


Figura 4.3: Distribuzione delle classi nel dataset LIS-Detection

4.4 Preprocessing e suddivisione dei dati

Prima di procedere con la fase di modellazione, il dataset è stato sottoposto a una serie di operazioni di preprocessing volte a garantire la coerenza e la qualità numerica dei campioni.

Poiché i dati risultano già normalizzati nell'intervallo $[0, 1]$, non è stato necessario applicare ulteriori procedure di scaling. Tuttavia, durante il caricamento è stato eseguito un controllo di consistenza per assicurarsi che ciascun vettore contenesse 42 valori validi e che non fossero presenti elementi mancanti o duplicati.

Tutti i campioni sono stati confermati corretti, in quanto derivano direttamente dall'output di MediaPipe, che garantisce un numero fisso di landmark per ogni acquisizione. Le etichette di classe sono state convertite da stringhe a valori numerici interi mediante un *label encoder*, così da consentirne la gestione da parte delle librerie di *machine learning*.

Successivamente, il dataset è stato suddiviso in tre sottoinsiemi indipendenti:

- *Training set (70%)*, destinato all'apprendimento dei modelli;
- *Validation set (15%)*, utilizzato per la calibrazione dei parametri;
- *Test set (15%)*, impiegato per la valutazione finale delle prestazioni.

La suddivisione è stata eseguita con criterio stratificato, mantenendo in ciascun sottoinsieme la stessa proporzione di esempi per ogni classe. Questo garantisce che ogni lettera dell'alfabeto sia rappresentata equamente in tutte le fasi del processo sperimentale, prevenendo bias e garantendo una valutazione equilibrata del sistema.

Il dataset preprocessato è stato quindi salvato in formato binario (*NumPy.npz*), in modo da poter essere facilmente ricaricato durante le successive fasi di progettazione e addestramento del classificatore neurale.

La procedura di suddivisione del dataset in sottoinsiemi è mostrata in Figura 4.4.

```
# === Split ===
X_train, X_temp, y_train, y_temp = train_test_split(*arrays: X, y_encoded, test_size=0.3, stratify=y_encoded, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(*arrays: X_temp, y_temp, test_size=0.5, stratify=y_temp, random_state=42)

np.savez(TEST_DATA_FILE, X_test=X_test, y_test=y_test, X_train=X_train)

print(f"[INFO] Train: {len(X_train)} | Val: {len(X_val)} | Test: {len(X_test)}")
```

Figura 4.4: Estratto di codice per la suddivisione stratificata del dataset in train, validation e test set.

Capitolo 5

Modellazione e training del classificatore neurale

5.1 Specifiche del modello e criteri di progettazione

La progettazione del classificatore neurale per il riconoscimento della Lingua dei Segni Italiana (LIS) è stata guidata da due obiettivi principali:

1. Ottenere un modello accurato, capace di distinguere correttamente le 26 lettere dell'alfabeto manuale LIS.
2. Mantenere un'elevata efficienza computazionale, in vista della successiva implementazione su microcontrollore STM32N6.

Considerando che ciascun campione del dataset è rappresentato da 42 feature numeriche, ovvero la rappresentazione delle coordinate normalizzate dei 21 landmark della mano, è stato naturale orientarsi verso una rete neurale completamente connessa (*feed-forward dense network*). Questa tipologia di architettura, pur essendo concettualmente più semplice rispetto a reti convoluzionali (CNN), è particolarmente adatta quando le feature in ingresso sono già strutturate e prive di informazione spaziale diretta, come nel caso dei landmark.

Il modello è stato sviluppato e addestrato in ambiente *TensorFlow/Keras*, per la sua flessibilità, la facilità di deployment e la compatibilità con strumenti di conversione come *STM32Cube.AI*.

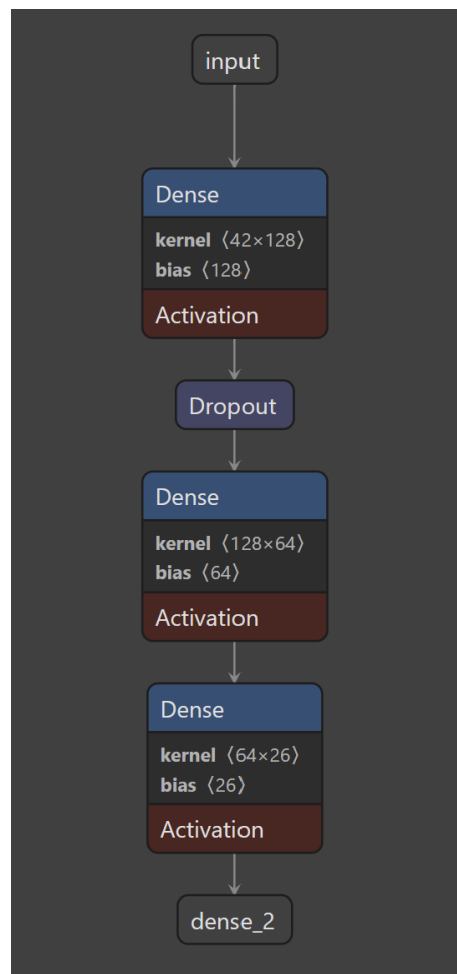
Le specifiche implementative prevedono un flusso completo: caricamento e preprocessing dei dati, codifica delle etichette, normalizzazione, suddivisione del dataset e definizione del modello neurale.

5.2 Architettura scelta e training

Il modello è stato implementato come una rete sequenziale composta da cinque strati principali, come riportato nella Figura 5.1. In particolare, la Figura 5.1a riporta la definizione del modello in ambiente TensorFlow/Keras, mentre la Figura 5.1b ne illustra la struttura grafica generata con Netron.

```
# === Definizione del modello ===  
model = tf.keras.Sequential([  
    tf.keras.layers.Input(shape=(42,), name="input"),  
    tf.keras.layers.Dense(128, activation="relu"),  
    tf.keras.layers.Dropout(0.3),  
    tf.keras.layers.Dense(64, activation="relu"),  
    tf.keras.layers.Dense(len(encoder.classes_), activation="softmax")  
])
```

(a) Definizione del modello neurale in TensorFlow/Keras.



(b) Rappresentazione grafica della rete completamente connessa (Netron).

Figura 5.1: Architettura del classificatore neurale per il riconoscimento delle lettere LIS: (a) definizione del modello in codice, (b) rappresentazione grafica della rete DNN.

L'architettura adotta una configurazione di tipo *Deep Neural Net-*

work (DNN) con:

- Uno *strato di input* di 42 neuroni (uno per ogni feature)
- Due *strati nascosti* completamente connessi (Dense), rispettivamente di 128 e 64 neuroni, con funzione di attivazione ReLU (Rectified Linear Unit), scelta per la sua efficienza e stabilità nel training
- Un *livello di dropout* intermedio con probabilità 0.3 per prevenire l'overfitting
- Uno *strato di output* con 26 neuroni e attivazione softmax, che produce una distribuzione di probabilità sulle 26 classi dell'alfabeto LIS

L'addestramento è stato effettuato utilizzando:

- *Funzione di perdita*: Sparse Categorical Cross-Entropy, appropriata per problemi di classificazione multiclasse con etichette intere
- *Ottimizzatore*: Adam, scelto per la sua capacità di adattare dinamicamente il tasso di apprendimento durante la fase di training
- *Metrica di valutazione*: Accuracy, ossia la percentuale di campioni classificati correttamente

L'addestramento è stato condotto per 30 epoche con un batch size di 32 campioni, bilanciando stabilità e rapidità di convergenza.

I dati di validazione (15% del totale) sono stati utilizzati per monitorare le prestazioni del modello e rilevare eventuali segnali di overfitting.

Durante il training, l'accuratezza è stata tracciata per ogni epoca (Fig.5.2), consentendo di analizzare visivamente l'andamento della convergenza.

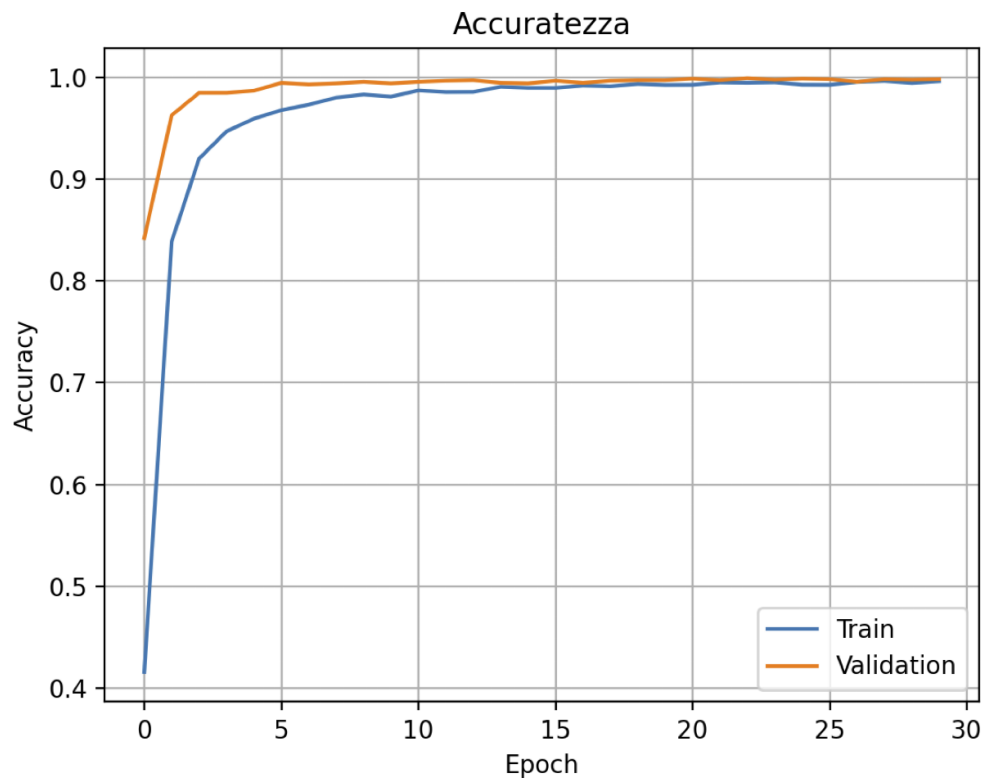


Figura 5.2: Andamento dell'accuratezza durante il training e la validazione del modello

Il modello mostra una crescita stabile dell'accuratezza di training con un andamento coerente su validation, indicando un buon equilibrio tra apprendimento e generalizzazione.

5.3 Metriche di valutazione: accuratezza e matrice di confusione

La valutazione del modello è stata condotta utilizzando il test set indipendente (15% dei campioni complessivi), mantenendo lo stesso bilanciamento delle classi osservato nel dataset originale.

Il principale indicatore utilizzato è stato l'accuratezza, definita come il rapporto tra il numero di previsioni corrette e il numero totale di campioni testati.

Il classificatore ha raggiunto un'accuratezza complessiva prossima al 99,9%, evidenziando un comportamento estremamente stabile e generalizzante.

Questo risultato conferma che la rete è riuscita ad apprendere in modo efficace la relazione tra le coordinate dei landmark e le corrispondenti lettere dell'alfabeto manuale LIS, senza manifestare fenomeni di overfitting o instabilità numerica. Per analizzare in maniera più dettagliata il comportamento del sistema su ciascuna classe, è stata calcolata la matrice di confusione, rappresentata in Figura 5.3.

Ogni cella (i,j) indica il numero di campioni appartenenti alla classe reale i che sono stati predetti come classe j .

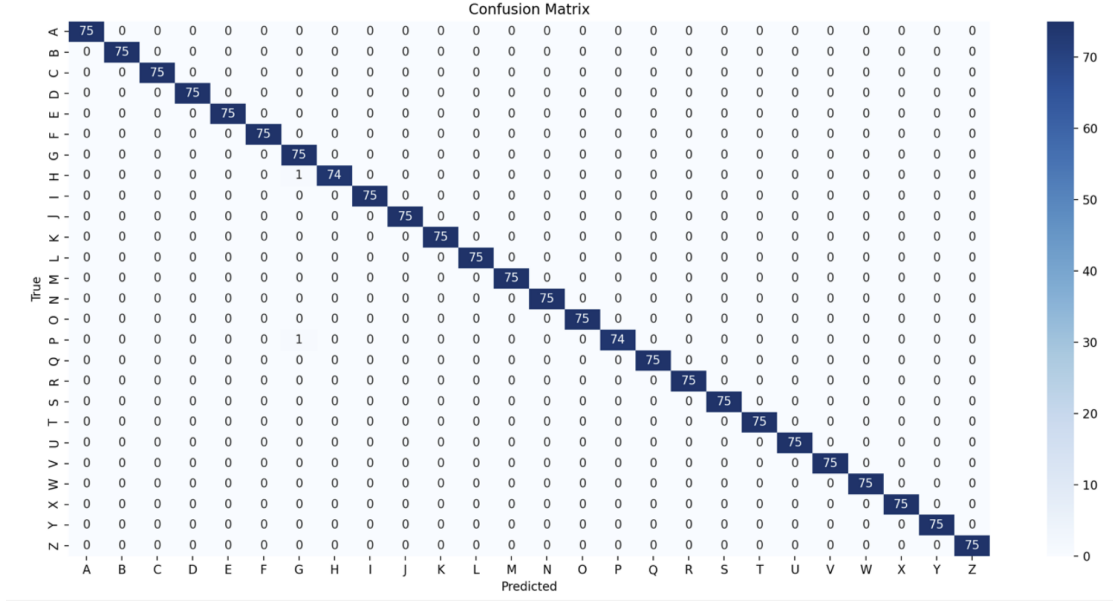


Figura 5.3: Matrice confusione

La Figura 5.3 mostra chiaramente una diagonale quasi perfetta, a testimonianza della capacità del modello di distinguere correttamente tutte le lettere del dataset.

Ogni classe contiene 75 campioni nel test set e per quasi tutte le lettere il modello ha ottenuto una classificazione esatta di tutti i 75 esempi. Solo due errori isolati sono stati registrati, un campione della classe H riconosciuto come P , e uno della classe P classificato come H , suggerendo una possibile somiglianza morfologica tra i rispettivi gesti o una lieve variazione posizionale nei landmark della mano.

Dal punto di vista pratico, questo livello di accuratezza rende il modello pienamente idoneo ad essere utilizzato come modulo di classificazione embedded, in grado di operare in tempo reale su microcontrollori a risorse limitate.

La Figura 5.4 riporta una sequenza di frame rappresentativi del

riconoscimento della parola “CIAO”, in cui ogni gesto corrisponde a una singola lettera dell’alfabeto manuale italiano. Si osserva come il sistema identifichi correttamente le lettere *C*, *I*, *A* e *O*, con probabilità prossime al 100%, evidenziando la capacità del modello di generalizzare correttamente anche in condizioni di illuminazione naturale e posizionamento non perfettamente centrato.



Figura 5.4: Sequenza di riconoscimento della parola “CIAO”: esempi di output del sistema durante l’inferenza in tempo reale. Si notano i landmark estratti da MediaPipe e la classificazione corretta di ciascun gesto con la relativa probabilità.

Questa dimostrazione visiva conferma la robustezza del classificatore e la sua idoneità ad essere utilizzato in applicazioni real-time di riconoscimento gestuale. La precisione quasi perfetta ottenuta in ambiente desktop costituisce il punto di partenza per la successiva

fase di porting del modello su piattaforma embedded, in cui l'obiettivo è garantire prestazioni comparabili anche in presenza di risorse computazionali fortemente limitate.

Capitolo 6

Implementazione hardware e validazione

Dopo la fase di validazione del modello in ambiente desktop, è stata progettata un'architettura di inferenza ibrida PC-MCU per testare il comportamento del classificatore in condizioni prossime a quelle di un sistema embedded reale.

L'obiettivo di questa configurazione è stato quello di verificare l'affidabilità e la stabilità del modello neurale quantizzato direttamente su microcontrollore, mantenendo al contempo una pipeline di acquisizione e pre-processing flessibile su PC.

6.1 Architettura hardware e vincoli computazionali

La Figura 6.1 illustra il flusso di elaborazione complessivo e la distribuzione funzionale tra la componente software eseguita su PC e quella implementata su board STM32N6.

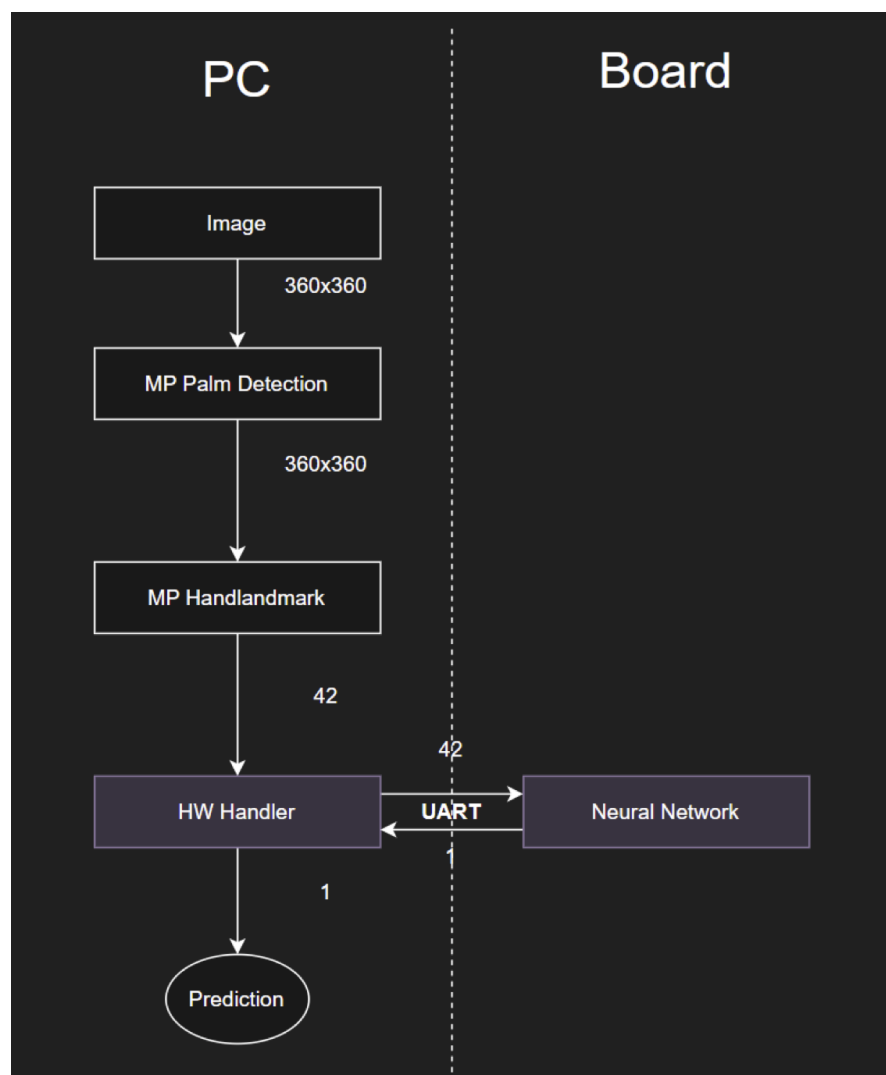


Figura 6.1: Architettura del sistema di riconoscimento LIS: pipeline di inferenza ibrida PC–STM32N6 con trasmissione dei vettori landmark via UART

Sul PC, il sistema acquisisce in tempo reale un frame video 360×360 pixel dalla webcam, sul quale viene applicata la pipeline di MediaPipe, composta dai moduli *Palm Detection* e *Hand Landmark Detection*. Il primo localizza la mano all'interno dell'immagine, mentre il secondo estrae i 21 punti di riferimento (landmark) che descrivono la posizione e l'orientamento delle dita e del palmo. Le coordinate normalizzate (x, y) vengono quindi combinate in un vettore di *42 feature*, che rappresenta la descrizione geometrica del gesto.

Tale vettore viene trasmesso tramite interfaccia *Universal Asynchronous Receiver-Transmitter (UART)* al microcontrollore attraverso un modulo software dedicato denominato *HW Handler*, responsabile della comunicazione seriale tra host e dispositivo.

Sulla board, il microcontrollore STM32N6 riceve il vettore e lo fornisce in input al modello neurale quantizzato, generato e ottimizzato con *STM32Cube.AI*. Il modello produce un singolo output numerico corrispondente alla *classe predetta*, cioè alla lettera dell'alfabeto manuale LIS riconosciuta.

Il risultato viene quindi inviato nuovamente al PC, dove il sistema visualizza la lettera riconosciuta insieme alle tre classi più probabili in sovrimpressione sul video.

Dal punto di vista tecnico, la progettazione è stata fortemente influenzata dai vincoli computazionali tipici dei sistemi embedded:

- *Memoria limitata*: richiede modelli compatti e quantizzati, ca-

paci di risiedere interamente nella memoria Flash e di utilizzare quantità ridotte di RAM

- *Basso consumo energetico*: il consumo di potenza elettrica deve rimanere basso (circa 200 mW per inferenza) per garantire l'autonomia in contesti portatili.
- *Latenza bassa*: il gesto deve essere riconosciuto quasi istantaneamente (entro 10 millisecondi), altrimenti l'interazione "tempo reale" non è naturale.

Tali vincoli hanno guidato tutte le fasi successive di conversione, ottimizzazione e validazione del modello neurale descritte nei paragrafi seguenti.

6.2 Conversione del modello in TensorFlow Lite

La prima fase del deployment ha riguardato la conversione del modello addestrato in ambiente TensorFlow in un formato compatibile con i dispositivi embedded. A tal fine, il modello in formato *.keras* è stato esportato in TensorFlow Lite (TFLite), un framework ottimizzato per l'inferenza su dispositivi a risorse limitate.

La conversione è stata effettuata mediante l'API `TFLiteConverter`, applicando una quantizzazione post-training in formato INT8, che ri-

duce sensibilmente la dimensione del modello e il consumo energetico, senza introdurre perdite significative di accuratezza. Durante questa fase, pesi e attivazioni sono stati scalati da rappresentazioni a 32 bit floating-point a rappresentazioni a 8 bit interi, ottenendo un miglior rapporto prestazioni/memoria.

Il risultato è un modello TFLite di dimensioni inferiori a 100 KB, perfettamente compatibile con la memoria Flash disponibile sulla STM32N6.

Per verificare che la conversione non avesse introdotto errori significativi, il modello TFLite è stato testato sullo stesso insieme di dati di validazione utilizzato per la versione Keras.

La *Figura 6.2* riporta il confronto diretto tra le accuratezze dei due modelli: la rete originale ha mantenuto un'accuratezza del *99,90%*, mentre la versione quantizzata ha ottenuto *99,33%*, con una perdita inferiore allo *0,6%*. Questa differenza minima conferma l'elevata stabilità numerica del modello anche dopo la riduzione di precisione.

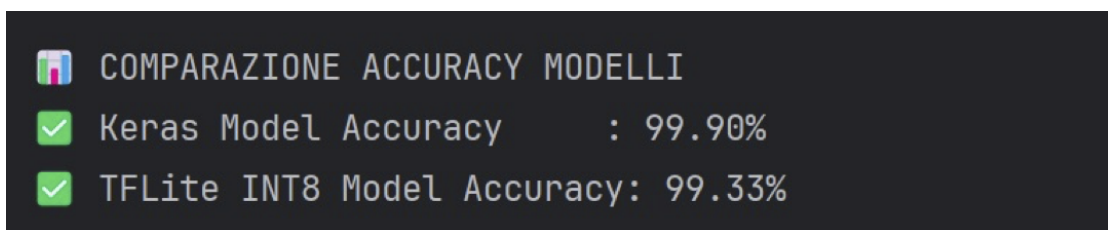


Figura 6.2: Confronto tra accuratezza del modello Keras originale e del modello quantizzato TensorFlow Lite INT8.

6.3 Integrazione con STM32Cube.AI e inferenza sul dispositivo

Dopo la validazione del modello in ambiente desktop e la sua conversione in formato TensorFlow Lite quantizzato, è stata avviata la fase di integrazione sul microcontrollore STM32N6 utilizzando l'ambiente di sviluppo STM32Cube.AI, fornito da STMicroelectronics.

Questo strumento rappresenta un componente essenziale dell'ecosistema STM32, poiché consente di importare modelli pre-addestrati e convertirli automaticamente in codice C altamente ottimizzato per l'esecuzione su architetture embedded dotate di acceleratori neurali integrati.

6.3.1 Analisi dell'allocazione della memoria

La Figura 6.3 riporta il riepilogo dell'utilizzo della memoria generato da STM32Cube.AI. È possibile osservare che il modello risiede interamente nella sezione di memoria *npuRAM5*, dove sono allocati sia i pesi che le attivazioni, per un totale di 16.511 kB. Tale valore corrisponde a circa il 3.69% della capacità disponibile per la NPU, mentre tutte le altre sezioni (*flexMEM*, *cpuRAM*, *hyperRAM*) risultano inutilizzate o destinate a funzioni di supporto, come la gestione dei buffer di input e output.

Memory usage information (input/output buffers are included in activations)				
flexMEM	[0x34000000 - 0x34000000]:	0 B /	0 B	(0.00 % used)
cpuRAM1	[0x34064000 - 0x34064000]:	0 B /	0 B	(0.00 % used)
cpuRAM2	[0x34100000 - 0x34200000]:	0 B /	1.000 MB	(0.00 % used)
npuRAM3	[0x34200000 - 0x34270000]:	0 B /	448.000 kB	(0.00 % used)
npuRAM4	[0x34270000 - 0x342E0000]:	0 B /	448.000 kB	(0.00 % used)
npuRAM5	[0x342E0000 - 0x34350000]:	16.511 kB /	448.000 kB	(3.69 % used)
npuRAM6	[0x34350000 - 0x343C0000]:	0 B /	448.000 kB	(0.00 % used)
octoFlash	[0x71000000 - 0x71000000]:	0 B /	0 B	(0.00 % used)
hyperRAM	[0x90000000 - 0x90000000]:	0 B /	0 B	(0.00 % used)

Total:			16.511 kB	

Figura 6.3: Report di utilizzo della memoria generato da STM32Cube.AI. Il modello risiede interamente nella memoria dedicata alla NPU, per un totale di 16.511 kB.

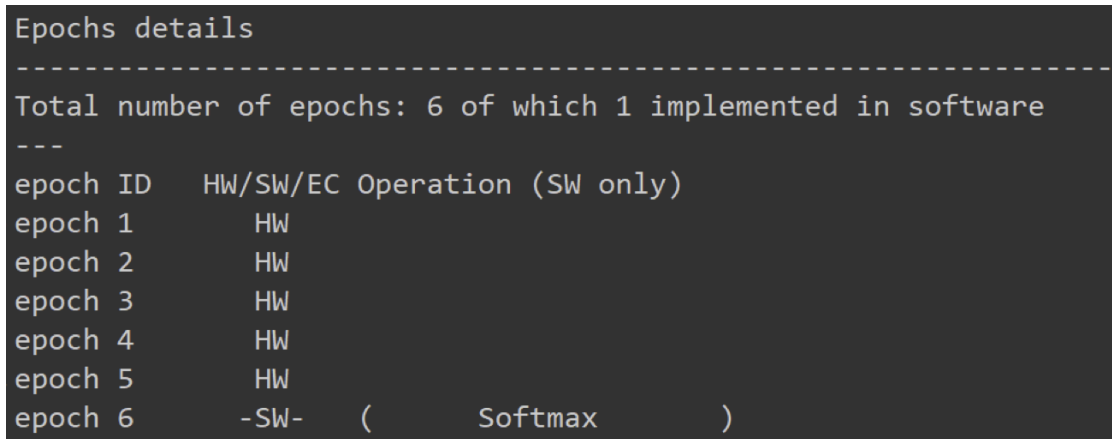
Questa distribuzione è particolarmente significativa poiché dimostra come l'intera rete neurale, comprensiva delle strutture di inferenza e dei dati di supporto, possa essere ospitata interamente nella memoria on-chip. Ciò consente di ridurre al minimo la latenza e il consumo energetico, evitando l'accesso a memorie esterne, tipicamente più lente e dispendiose dal punto di vista energetico.

La gestione completamente locale dei pesi rappresenta inoltre un importante vantaggio in termini di sicurezza e affidabilità, poiché elimina la necessità di trasferimenti dati esterni durante il runtime

6.3.2 Distribuzione delle epoche tra hardware e software

L'analisi dell'esecuzione del modello, riportata in Figura 6.4, evidenzia la suddivisione delle diverse fasi di calcolo tra le componenti *hardware* e *software* del sistema.

In particolare, il report mostra che il processo di inferenza è articolato in sei epoche operative, di cui cinque vengono eseguite interamente in hardware (HW) sull'acceleratore neurale *Neural-ART™*, mentre l'ultima, corrispondente alla funzione di attivazione *Softmax*, viene gestita in software (SW) dal core *ARM Cortex-M55*.



Epochs details			

Total number of epochs: 6 of which 1 implemented in software			

epoch	ID	HW/SW/EC	Operation (SW only)
epoch 1		HW	
epoch 2		HW	
epoch 3		HW	
epoch 4		HW	
epoch 5		HW	
epoch 6		-SW-	(Softmax)

Figura 6.4: Dettaglio della distribuzione delle epoche di esecuzione: cinque blocchi gestiti dalla NPU e un'ultima operazione di tipo Softmax eseguita in software.

Questa architettura ibrida riflette un bilanciamento ottimale tra efficienza e flessibilità.

Le operazioni più onerose dal punto di vista computazionale, come i layer *fully connected*, caratterizzati da un elevato numero di moltiplicazioni e somme, vengono eseguite direttamente dalla NPU, che grazie alla sua struttura parallela è in grado di gestire simultaneamente un grande numero di operazioni vettoriali con un consumo energetico minimo.

La CPU, al contrario, entra in gioco soltanto nella fase conclusiva dell'inferenza, dove si occupa di normalizzare le probabilità e

determinare la classe con la massima confidenza.

Questa sinergia tra core general-purpose e acceleratore dedicato permette di ridurre drasticamente i tempi di elaborazione, completando l'intero processo di inferenza in meno di *10 millisecondi*.

Allo stesso tempo, il consumo energetico rimane estremamente contenuto, circa 200 mW durante l'inferenza rendendo il sistema perfettamente compatibile con i requisiti di efficienza e autonomia tipici degli ambienti *TinyML*.

Capitolo 7

Implementazione software

Dopo l'integrazione del modello neurale quantizzato nel microcontrollore STM32N6, l'ultima fase del lavoro ha riguardato lo sviluppo della parte software di coordinamento e validazione del sistema di predizione. Questa fase è stata fondamentale per trasformare il modello embedded in un sistema realmente operativo, capace di eseguire in modo continuo il riconoscimento dei gesti e di comunicare i risultati in tempo reale con l'ambiente esterno.

L'obiettivo principale è stato quello di collegare la pipeline di elaborazione visiva, eseguita su PC, con l'inferenza embedded sul microcontrollore, realizzando un flusso completo di riconoscimento gestuale che fosse robusto, reattivo e sincronizzato.

7.1 Struttura generale e obiettivi funzionali

Il software sviluppato rappresenta il collegamento logico tra le due componenti principali del sistema:

- Il *PC* che esegue la pipeline di *MediaPipe*, ed elabora le immagini acquisite dalla webcam e calcola le coordinate dei landmark della mano
- Il microcontrollore *STM32N6*, che riceve i dati pre-processati e li utilizza per eseguire l'inferenza del modello neurale quantizzato.

Per realizzare questo collegamento è stato sviluppato un modulo dedicato, denominato *HW_Handler*, responsabile di gestire la comunicazione seriale tra PC e board. Esso si occupa della trasmissione dei dati, dell'avvio dell'inferenza e della ricezione dei risultati, fungendo da intermediario tra il livello software di alto livello (Python) e il livello firmware a basso livello (C su STM32).

All'interno del codice Python, questo ruolo è svolto dalla funzione *perform_prediction()*, che rappresenta a tutti gli effetti l'equivalente logico dell'*HW_Handler*.

Inoltre, la funzione è progettata in modo da poter operare in due modalità complementari:

- *Locale*, utilizzando la versione quantizzata del modello *TensorFlow Lite* direttamente sul PC, per verificare la correttezza numerica e le prestazioni del modello.
- *Remota*, delegando il calcolo al microcontrollore, per validare il comportamento reale del sistema embedded.

Questa doppia modalità di esecuzione garantisce coerenza tra le due versioni del modello e consente di verificare facilmente la correttezza della quantizzazione e del trasferimento dei dati, evitando discrepanze tra la fase di simulazione e quella di deployment.

7.2 La funzione `perform_prediction()`

La funzione *perform_prediction()* rappresenta il cuore logico del sistema software di comunicazione tra PC e microcontrollore. Essa incarna, a livello operativo, il comportamento dell'*HW_Handler*, coordinando tutte le fasi necessarie all'esecuzione di una predizione: dalla preparazione del vettore di input alla ricezione del risultato elaborato dalla board STM32N6.

Dal punto di vista concettuale, *perform_prediction()* può essere vista come un ponte bidirezionale tra due domini distinti ma complementari:

- *Il dominio software*, in cui avviene l'acquisizione e il pre-processing delle immagini tramite MediaPipe e la preparazione dei dati di input per la rete neurale;
- *Il dominio embedded*, dove il modello quantizzato viene eseguito in tempo reale sul microcontrollore STM32N6 mediante l'ambiente STM32Cube.AI.

L'intero processo di predizione è organizzato come una sequenza ciclica composta da quattro fasi principali:

1. *Allocazione e preparazione dell'input*

Inizializzazione del modello quantizzato e conversione dei dati in formato compatibile (*int8*).

2. *Serializzazione e trasmissione*

Invio del vettore di feature al microcontrollore attraverso l'interfaccia UART.

3. *Inferenza hardware*

Elaborazione del vettore in ambiente STM32Cube.AI e generazione dell'output di rete.

4. *Ricezione e decodifica*

Lettura dei risultati da parte del PC e ricostruzione della predizione finale

In Figura 7.1 è mostrato il flusso di comunicazione tra PC e microcontrollore STM32N6.

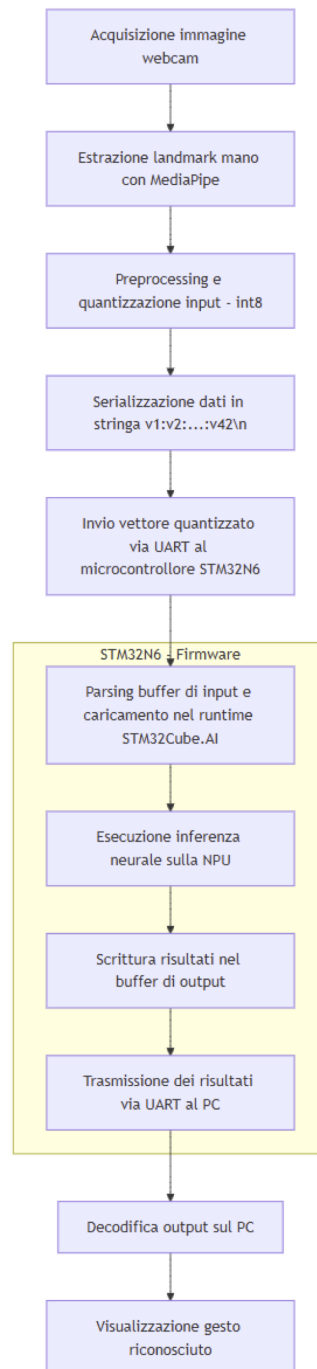


Figura 7.1: Flusso di comunicazione tra PC e microcontrollore STM32N6.

Nelle sezioni seguenti vengono illustrate nel dettaglio le diverse fasi di funzionamento della funzione `perform_prediction()`, ciascuna delle quali corrisponde a un passo della pipeline di predizione, dal preprocessing alla validazione dei risultati.

7.2.1 Allocazione delle risorse e preparazione dell'input

All'inizio della funzione, vengono allocate le risorse necessarie all'esecuzione del modello quantizzato TensorFlow Lite (Figura 7.2)

```
self.classifier_quant.allocate_tensors()
input_details = self.classifier_quant.get_input_details()
output_details = self.classifier_quant.get_output_details()
```

```
in_scale, in_zero_point = input_details[0]["quantization"]
in_buffer_int8 = (in_buffer/in_scale + in_zero_point).astype(np.int8)
```

Figura 7.2: Inizializzazione del modello quantizzato e conversione dell'input in formato int8

Queste istruzioni inizializzano la rete neurale e recuperano le informazioni relative ai tensori di input e output, come la scala di quantizzazione (*scale*) e il punto zero (*zero_point*). Tali parametri vengono utilizzati per convertire i valori reali dei landmark, ottenuti da MediaPipe, nel formato intero a 8 bit richiesto dal modello quantizzato:

La quantizzazione è una trasformazione fondamentale che consente di ridurre la dimensione dei dati e il consumo di memoria, mantenendo

un'accuratezza pressoché invariata. Essa converte i valori reali in virgola mobile (x_{float}) in numeri interi a 8 bit (x_{int8}), secondo la relazione mostrata nella equazione 7.1.

$$x_{\text{int8}} = \frac{x_{\text{float}}}{\text{scale}} + \text{zero_point} \quad (7.1)$$

Attraverso l'equazione 7.1, ciascun landmark della mano (costituito da coordinate normalizzate tra 0 e 1) viene convertito in un valore discreto compreso tra -128 e $+127$. Il risultato finale è un vettore `int8` di 42 elementi, pronto per essere inviato al microcontrollore.

Questa fase, apparentemente semplice, è cruciale per garantire la coerenza numerica tra il modello eseguito su PC e quello deployato sul dispositivo embedded.

7.2.2 Serializzazione e invio del vettore di input

Una volta ottenuto il vettore quantizzato, i dati devono essere trasmessi alla board STM32N6.

Per semplificare la comunicazione e rendere i pacchetti facilmente interpretabili, i valori vengono serializzati in formato testuale: ogni numero è separato dal carattere “:” e la stringa termina con un carattere di nuova linea ‘`\n`’, che funge da delimitatore per la board. Come mostrato in Figura 7.3, il frammento di codice Python esegue la serializzazione del vettore quantizzato e lo invia tramite interfaccia UART al microcontrollore.

```
message = ":".join(str(x) for x in in_buffer_int8[0]) + "\n"
print("Sending:", message)
```

Figura 7.3: Serializzazione e invio vettore quantizzato (UART)

L'invio avviene tramite la porta seriale UART, gestita dall'oggetto *self.ser*, che rappresenta la connessione attiva tra PC e microcontrollore. Dopo la trasmissione, viene inserita una breve attesa (*time.sleep(0.1)*) per permettere alla board di completare la fase di ricezione e avvio dell'inferenza (Figura 7.4). Questa scelta evita conflitti o perdite di pacchetti, garantendo la sincronizzazione tra i due ambienti.

```
self.ser = serial.Serial(
    port="COM3",
    baudrate=115200,
    timeout=3
)
```

```
# --- Invia i dati ---
self.ser.write(message.encode("utf-8"))

# --- Delay per dare tempo alla board di elaborare ---
time.sleep(0.1)
```

Figura 7.4: Trasmissione seriale del vettore quantizzato.

7.2.3 Ricezione e decodifica della risposta

Una volta completata l'inferenza, il microcontrollore restituisce una stringa contenente i valori di output, ovvero le probabilità stimate dal modello per ciascuna classe dell'alfabeto LIS.

Il PC legge la risposta e la decodifica, ricostruendo la lista numerica dei valori ricevuti (Figura 7.5)

```
response = self.ser.read_all().decode(errors='ignore').strip()

probs = []
if response:
    print("Received from STM32:")
    probs = [int(x) for x in response.split(':') if x]
    print(probs)
else:
    print("Nessuna risposta ricevuta dalla STM32")
```

Figura 7.5: Ricezione e parsing dell'output del microcontrollore

7.2.4 Inferenza locale per la validazione

Oltre a interfacciarsi con la board, la funzione *perform_prediction()* può eseguire la stessa inferenza anche in modalità locale, direttamente su PC.

In questa modalità, lo stesso modello quantizzato TensorFlow Lite viene eseguito direttamente sul PC, consentendo di confrontare i risultati con quelli ottenuti sul microcontrollore.

Come mostrato in Figura 7.6, il codice Python esegue il caricamento del modello quantizzato, l'inferenza locale e la lettura dei tensori di output per il confronto con i risultati ottenuti sul microcontrollore.

```
self.classifier_quant.set_tensor(input_details[0]["index"], in_buffer_int8)
self.classifier_quant.invoke()
out_buffer = self.classifier_quant.get_tensor(output_details[0]["index"])
```

Figura 7.6: Codice Python per l'esecuzione locale del modello quantizzato.

In questo modo è possibile confrontare, a parità di input, l'output prodotto in locale e quello restituito dal microcontrollore, verificando che la quantizzazione e la comunicazione non abbiano introdotto differenze significative.

7.3 Firmware e gestione dell'inferenza su STM32N6

Il firmware eseguito sul microcontrollore STM32N6 rappresenta la controparte hardware della funzione *perform_prediction()* vista nel lato PC.

Il suo ruolo è quello di ricevere i dati di input, prepararli per il modello neurale, eseguire l'inferenza tramite STM32Cube.AI e infine ritrasmettere i risultati al computer, completando il ciclo di predizione.

Questo componente è stata sviluppata in linguaggio C all'interno dell'ambiente STM32CubeIDE, e utilizza il runtime STM32Cube.AI, un motore ottimizzato che consente di eseguire modelli neurali quantizzati direttamente su microcontrollori a risorse limitate.

L'architettura complessiva segue un approccio *event-driven*, basato su interrupt e flag, che garantisce una comunicazione efficiente e non bloccante tra PC e microcontrollore.

Il comportamento generale del microcontrollore è regolato dal ciclo principale (*main()*), che gestisce tutte le fasi operative del sistema:

inizializzazione, attesa dei dati, inferenza e trasmissione del risultato.

7.3.1 Gestione della comunicazione UART

La comunicazione tra PC e microcontrollore avviene attraverso l'interfaccia UART, scelta per la sua semplicità e affidabilità nei sistemi embedded.

Questo canale seriale permette di scambiare i dati tra i due dispositivi in modo lineare e continuo, garantendo un flusso stabile anche durante l'elaborazione in tempo reale. Il PC invia periodicamente una stringa contenente i 42 valori quantizzati, ciascuno corrispondente alle coordinate dei landmark della mano estratti da MediaPipe.

Sul microcontrollore STM32N6, la ricezione dei dati è gestita da una callback asincrona, mostrata in Figura 7.7, che viene automaticamente richiamata quando un nuovo carattere è disponibile.

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == LPUART1)
    {
        if (rx_data == '\n') {
            rx_buffer[rx_index] = '\0'; // chiudo la stringa

            flag_feature_received = 1;

            // Azzero l'indice per la prossima stringa
            rx_index = 0;
        }
        else {
            if (rx_index < sizeof(rx_buffer) - 1) {
                rx_buffer[rx_index++] = rx_data;
            }
        }

        HAL_UART_Receive_IT(&hlpuart1, &rx_data, 1);
    }
}
```

Figura 7.7: Callback di ricezione UART.

All'interno di questa funzione, ogni byte ricevuto viene salvato in un buffer temporaneo (`rx_buffer`). Quando il carattere ricevuto è un carattere di nuova linea (`\n`), la stringa viene considerata completa: il firmware la chiude correttamente, imposta il flag *flag_feature_received* = 1, e azzera l'indice di scrittura del buffer per prepararsi alla successiva trasmissione.

Questo semplice meccanismo consente al microcontrollore di riconoscere automaticamente quando un pacchetto è stato interamente ricevuto, senza necessità di handshake complessi.

Grazie alla gestione *interrupt-driven*, il microcontrollore non resta mai bloccato in attesa di dati. Mentre la UART riceve un messaggio, il

resto del programma può continuare ad eseguire altre operazioni, rendendo l'intero sistema più fluido e reattivo, un aspetto fondamentale in applicazioni real-time come il riconoscimento gestuale.

7.3.2 Ciclo operativo e parsing dell'input

Dopo l'inizializzazione delle periferiche e l'attivazione della ricezione asincrona, il microcontrollore entra nel suo ciclo operativo principale.

All'interno di questo loop, l'elaborazione è scandita da eventi: ogni volta che viene ricevuto un pacchetto completo di dati, il flag *flag_feature_received* segnala al firmware di procedere con la fase di inferenza.

Quindi, quando il flag viene impostato, il firmware sa che un set completo di 42 coordinate, corrispondenti ai landmark della mano estratti da MediaPipe, è pronto per essere elaborato.

Una volta ricevuto un pacchetto valido, il microcontrollore esegue la funzione *parse_int8_buffer()*, che converte la stringa di caratteri inviata dal PC in un vettore di interi *int8_t*. Questa conversione è fondamentale perché il modello integrato nella board lavora interamente in formato quantizzato, ossia con valori numerici a 8 bit.

In questo modo si riducono al minimo l'uso di memoria e i tempi di calcolo, senza compromettere l'accuratezza.

Come mostrato in Figura 7.8, il firmware, una volta ricevuto il pacchetto di dati, verifica il flag di ricezione e prepara i buffer di input

e output necessari per l'esecuzione dell'inferenza neurale.

```
// Se abbiamo ricevuto 42 feature dal PC (flag alzato)
if (flag_feature_received)
{
    flag_feature_received = 0;

    //HAL_UART_Transmit(&hlpuart1, (uint8_t*)rx_buffer, strlen(rx_buffer), HAL_MAX_DELAY);

    int8_t *buffer_in;
    int8_t *buffer_out;

    LL_ATON_RT_RetValues_t ll_aton_rt_ret = LL_ATON_RT_DONE;
    const LL_Buffer_InfoTypeDef * ibuffersInfos = NN_Interface_Default.input_buffers_info();
    const LL_Buffer_InfoTypeDef * obuffersInfos = NN_Interface_Default.output_buffers_info();
    buffer_in = (int8_t *)LL_Buffer_addr_start(&ibuffersInfos[0]);
    buffer_out = (int8_t *)LL_Buffer_addr_start(&obuffersInfos[0]);

    //int8_t feature_buffer[NUM_FEATURES];
    int n = parse_int8_buffer(rx_buffer, buffer_in, NUM_FEATURES);
}
```

Figura 7.8: Gestione del flag di ricezione e preparazione dei buffer di input e output per l'inferenza neurale.

I dati convertiti vengono poi copiati nel buffer di input del runtime STM32Cube.AI, che gestisce internamente la memoria riservata al modello neurale.

7.3.3 Esecuzione dell'inferenza neurale

Una volta ricevuti e convertiti i dati di input, il microcontrollore procede con l'esecuzione del modello neurale quantizzato. Questa fase rappresenta il cuore computazionale del firmware e sfrutta appieno le potenzialità della *Neural Processing Unit (NPU)* integrata nella STM32N6.

L'inferenza viene avviata inizializzando il runtime di STM32Cube.AI attraverso le funzioni in Figura 7.9 che preparano le strutture da-

ti necessarie, allocano i buffer e caricano in memoria la rete neurale pre-compilata.

```
LL_ATON_RT_RuntimeInit();  
LL_ATON_RT_Init_Network(&NN_Instance_Default);
```

Figura 7.9: Inizializzazione del runtime STM32Cube.AI e caricamento della rete neurale quantizzata nella memoria del microcontrollore.

Successivamente, il calcolo vero e proprio è eseguito tramite la funzione `LL_ATON_RT_RunEpochBlock(&NN_Instance_Default)`, come mostrato in Figura 7.10, che richiama la NPU per eseguire la sequenza di operazioni matematiche alla base del modello.

```
do {  
    /* Execute first/next step */  
    ll_aton_rt_ret = LL_ATON_RT_RunEpochBlock(&NN_Instance_Default);  
    /* Wait for next event */  
    if (ll_aton_rt_ret == LL_ATON_RT_WFE) {  
        LL_ATON_OSAL_WFE();  
    }  
} while (ll_aton_rt_ret != LL_ATON_RT_DONE);
```

Figura 7.10: Loop di esecuzione dell'inferenza con la funzione `LL_ATON_RT_RunEpochBlock()`.

Durante l'esecuzione, la CPU principale (ARM Cortex-M55) assume un ruolo di supervisione, sincronizzando l'attività della NPU e gestendo eventuali operazioni non supportate nativamente, come la funzione di attivazione *Softmax*. Questo meccanismo di cooperazione tra CPU e acceleratore consente di ottenere una latenza inferiore a 10 ms per ogni inferenza, mantenendo al contempo un consumo energetico ridotto.

Al termine dell'elaborazione, i risultati vengono salvati nel buffer di output (`buffer_out`), dove ciascun elemento rappresenta la probabilità che il gesto corrisponda a una determinata classe dell'alfabeto LIS.

Una volta completata l'inferenza, il contesto di rete viene deiniziato come mostrato in Figura 7.11, in modo da liberare le risorse e consentire una nuova predizione al ciclo successivo.

```
LL_ATON_RT_DeInit_Network(&NN_Instance_Default);  
LL_ATON_RT_RuntimeDeInit();
```

Figura 7.11: Deiniziallizzazione del contesto di rete.

7.3.4 Trasmissione del risultato

Dopo l'inferenza, i valori contenuti nel buffer di output vengono inviati al PC per la ricostruzione della predizione finale, come mostrato in Figura 7.12.

```
int num_values = LL_Buffer_len(&obuffersInfos[0]);  
  
// Trasmetti tutti i valori int8  
for (int i = 0; i < num_values; i++) {  
    char temp[16];  
    snprintf(temp, sizeof(temp), "%d:", buffer_out[i]);  
    HAL_UART_Transmit(&hlpuart1, (uint8_t*)temp, strlen(temp), HAL_MAX_DELAY);  
}  
HAL_UART_Transmit(&hlpuart1, (uint8_t*)"\r\n", 2, HAL_MAX_DELAY);
```

Figura 7.12: Estratto di codice relativo alla trasmissione dell'output al PC. Ogni valore del buffer viene convertito in stringa e inviato tramite UART, seguito da un carattere di nuova linea.

Il frammento di codice riportato in Figura 7.12 mostra la fase finale del processo di inferenza, in cui i risultati calcolati dal modello

neurale vengono trasmessi al PC. Dopo aver determinato il numero di valori di output (*num_values*), il firmware scorre il vettore *buffer_out*, convertendo ogni elemento in stringa mediante la funzione *snprintf()* e inviandolo sulla porta seriale con *HAL_UART_Transmit()*. I valori vengono separati dal carattere “:” e il pacchetto si conclude con la sequenza “\r\n”, che indica la fine della trasmissione.

Questo approccio testuale, sebbene semplice, garantisce massima compatibilità e leggibilità nella comunicazione con il software Python, che può così ricostruire facilmente il vettore numerico ricevuto. Inoltre, l’uso di un formato leggibile facilita notevolmente le fasi di debug e validazione del sistema durante i test.

In questo modo, il ciclo di inferenza si completa: il microcontrollore esegue il modello, invia i risultati al PC e il software Python li traduce in una predizione leggibile.

L’integrazione tra hardware e software permette quindi di realizzare un sistema di riconoscimento gestuale realmente embedded, capace di operare in tempo reale con risorse computazionali estremamente ridotte.

Conclusioni

Il lavoro presentato ha dimostrato la fattibilità di un sistema di riconoscimento automatico della Lingua dei Segni Italiana basato su modelli di deep learning leggeri, ottimizzati per l'esecuzione in ambiente embedded.

A partire dall'acquisizione dei landmark tramite MediaPipe Hands, è stato sviluppato un modello neurale quantizzato capace di classificare in tempo reale i gesti corrispondenti alle lettere dell'alfabeto manuale LIS, con un'accuratezza prossima al *99,9%*.

L'integrazione del modello nel microcontrollore STM32N6, mediante l'ambiente *STM32Cube.AI*, ha confermato la possibilità di eseguire l'inferenza localmente con latenze inferiori a 10 ms e un consumo di potenza elettrica dell'ordine dei 200 mW, validando l'efficacia del paradigma *TinyML* per applicazioni di accessibilità e inclusione.

Dal punto di vista architetturale, il sistema si basa attualmente su una pipeline ibrida *PC-MCU*, nella quale il PC gestisce le fasi di visione artificiale (Palm Detection e Hand Landmark Detection) e pre-processing dei dati, mentre la board STM32N6 esegue la fase

di inferenza del classificatore neurale. Tale configurazione ha consentito di sperimentare in modo controllato la comunicazione hardware–software e di ottimizzare il flusso di predizione, coordinato dalla funzione *perform_prediction()*, che costituisce l’equivalente software dell’*HW_Handler* illustrato nel Capitolo 7.

Gli sviluppi futuri del progetto sono orientati verso una implementazione completamente hardware, nella quale tutte le fasi della pipeline, dall’acquisizione dell’immagine alla predizione finale, verranno eseguite direttamente sul microcontrollore.

In questa prospettiva, il PC non sarà più necessario, poiché l’intero flusso di elaborazione sarà gestito in modo autonomo dalla board, garantendo un funzionamento completamente offline, come mostrato in figura C.1

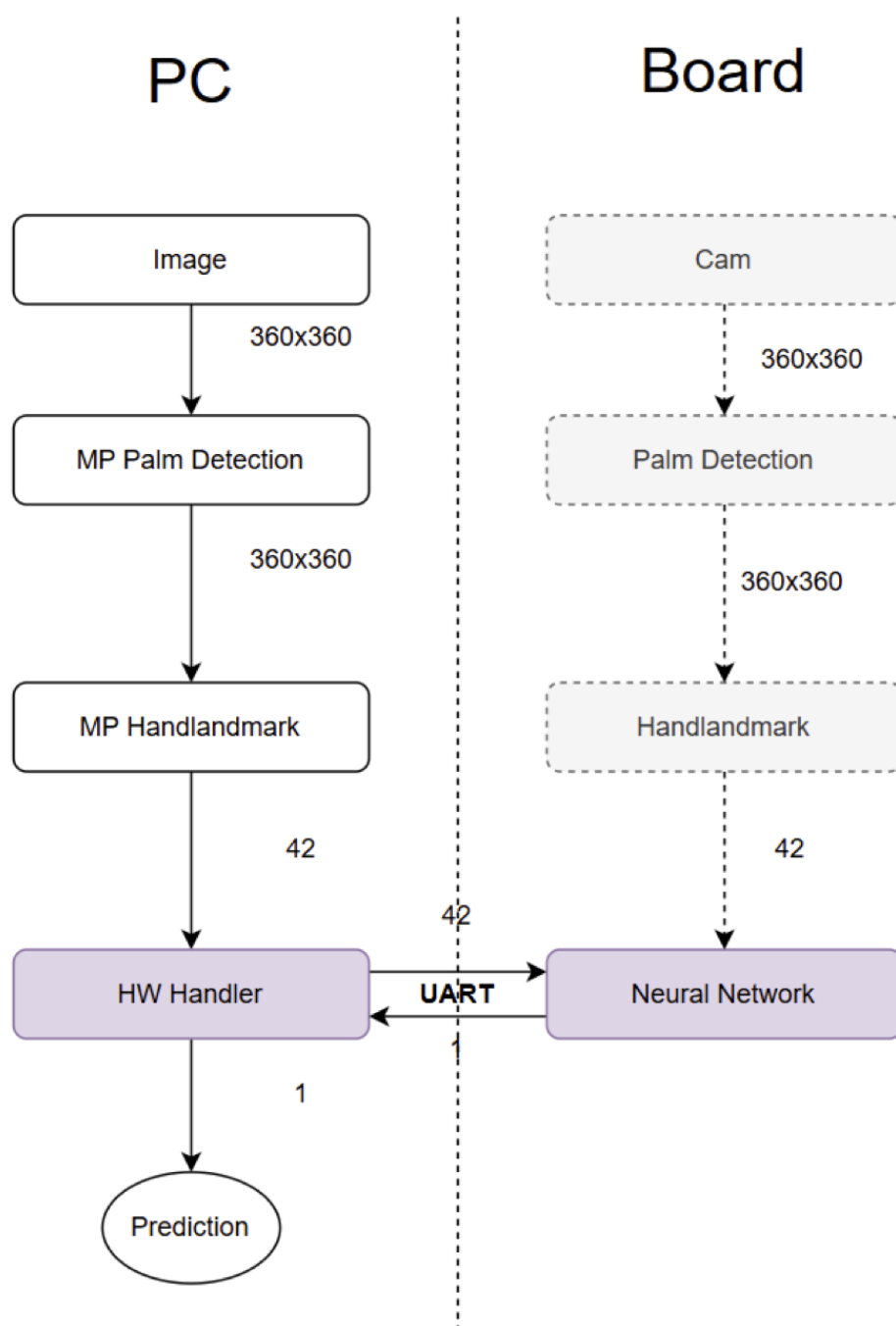


Figura C.1: Architettura del sistema completamente implementata su microcontrollore STM32N6.

In questa prospettiva, la webcam integrata della board sostituirà la fase di acquisizione gestita dal PC, mentre i moduli di Palm Detection e Hand Landmark Detection verranno portati in locale grazie all'impie-

go di modelli compatibili in formato *TensorFlow Lite*, già identificati durante la ricerca.

In particolare, sono stati individuati modelli pre-ottimizzati dal repository *PINTO00309/PINTO_model_zoo – Hand Detection and Tracking*, che offrono versioni *tf.lite* leggere e pienamente compatibili con le MCU STM32N6.

L'adozione di tali modelli permetterà di sostituire progressivamente le componenti *MediaPipe* con reti neurali completamente eseguibili in locale, eliminando la dipendenza dal PC e consentendo il riconoscimento totalmente offline.

Bibliografia

- [1] Ethnologue, “Languages of the World – database linguistico internazionale.” Pagina web.
- [2] Wikipedia, “List of sign languages by number of native signers.” Pagina web.
- [3] H. L. Lane, *When the Mind Hears: A History of the Deaf*. Random House, 1984.
- [4] M. C. Corballis, *From hand to mouth: The origins of language*. Princeton University Press, 2002.
- [5] T. Yang, X. Fan, B. Hou, J. Wang, and X. Chen, “Linguistic network in early deaf individuals: A neuroimaging meta-analysis,” *NeuroImage*, vol. 299, p. 120720, 2024.
- [6] G. Rizzolatti and L. Craighero, “The mirror-neuron system,” *Annu. Rev. Neurosci.*, vol. 27, no. 1, pp. 169–192, 2004.

- [7] H. B. Caldwell, “Sign and spoken language processing differences in the brain: A brief review of recent research,” *Annals of Neurosciences*, vol. 29, no. 1, pp. 62–70, 2022.
- [8] L. A. Petitto, R. J. Zatorre, K. Gauna, E. J. Nikelski, D. Dostie, and A. C. Evans, “Speech-like cerebral activity in profoundly deaf people processing signed languages: implications for the neural basis of human language,” *Proceedings of the National Academy of Sciences*, vol. 97, no. 25, pp. 13961–13966, 2000.
- [9] Aristotele, *Trattati sull’Anima (De Anima, De Sensu et Sensibili) e Storia degli Animali (Historia Animalium)*.
- [10] Narrare di Storia, “Storia di una lingua mai parlata: la lingua dei segni.” Pagina web, 2021.
- [11] J. P. Bonet, “Reducción de las letras y arte para enseñar a hablar a los mudos. madrid: Francisco abarca de angulo. recuperado de biblioteca digital hispánica. biblioteca nacional de españa.”
- [12] G. Bernini, “Russo cardona, tommaso/volterra, virgina, le lingue dei segni. storia e semiotica, carocci, roma 2007,” *LINGUISTICA E FILOLOGIA*, vol. 24, pp. 210–213, 2007.
- [13] Storia dei Sordi, “Il congresso di milano del 1880.” Pagina web.

- [14] W. C. Stokoe, *Sign Language Structure: An Outline of the Visual Communication Systems of the American Deaf*. University of Buffalo, 1960.
- [15] Ente Nazionale Sordi (ENS), “Materiali storici e informativi sulla diffusione della lis e l’opera di magarotto.” Documentazione istituzionale.
- [16] Camera dei Deputati, “Articolo 34-ter: Misure per il riconoscimento della lingua dei segni italiana e l’inclusione delle persone con disabilità uditiva.” Documento istituzionale (OCD177-5446), 2022.
- [17] S. La Grutta, M. A. Piombo, V. Spicuzza, M. Riolo, I. Fanara, E. Trombini, F. Andrei, and M. S. Epifanio, “The relationship between knowing sign language and quality of life among italian people who are deaf: A cross-sectional study,” in *Healthcare*, vol. 11, p. 1021, MDPI, 2023.
- [18] O. M. Al-Hassan, K. E. Bani-Hani, and M. M. Al-Masa’deh, “Sign language as a means of inclusion: A case study,” *International Journal of Disability, Development and Education*, vol. 71, no. 6, pp. 987–1001, 2024.
- [19] M. Marcuzzo, *La LIS come mezzo di inclusione e accessibilità*. Tesi di laurea magistrale, Università Ca’ Foscari Venezia, 2023.

- [20] E. D. Thoutenhoofd, “The sign language interpreter in inclusive education: Power of authority and limits of objectivism,” *The Translator*, vol. 11, no. 2, pp. 237–258, 2005.
- [21] L. Ulrich, G. Carmassi, P. Garelli, G. Lo Presti, G. Ramondetti, G. Marullo, C. Innocente, and E. Vezzetti, “Signify: leveraging machine learning and gesture recognition for sign language teaching through a serious game,” *Future Internet*, vol. 16, no. 12, p. 447, 2024.
- [22] M. S. Amin, S. T. H. Rizvi, and M. M. Hossain, “A comparative review on applications of different sensors for sign language recognition,” *Journal of Imaging*, vol. 8, no. 4, p. 98, 2022.
- [23] Ray-Ban, “Discover meta ray-ban display.” Pagina web aziendale.
- [24] G. Caligiore, R. Mineo, C. Spampinato, E. Ragonese, S. Palazzo, and S. Fontana, “Multisource approaches to italian sign language (lis) recognition: Insights from the multimedalis dataset,” in *Proceedings of the 10th Italian Conference on Computational Linguistics (CLiC-it 2024)*, pp. 132–140, 2024.
- [25] F. M. Najib, “Sign language interpretation using machine learning and artificial intelligence,” *Neural Computing and Applications*, vol. 37, no. 2, pp. 841–857, 2025.

- [26] J. Qi, L. Ma, Z. Cui, and Y. Yu, “Computer vision-based hand gesture recognition for human-robot interaction: a review,” *Complex & Intelligent Systems*, vol. 10, no. 1, pp. 1581–1606, 2024.
- [27] E. Trucco and A. Verri, *Introductory techniques for 3-D computer vision*, vol. 201. Prentice Hall Englewood Cliffs, 1998.
- [28] M. A. Ahmed, B. B. Zaidan, A. A. Zaidan, M. M. Salih, and M. M. B. Lakulu, “A review on systems-based sensory gloves for sign language recognition state of the art between 2007 and 2017,” *Sensors*, vol. 18, no. 7, p. 2208, 2018.
- [29] Z. R. Saeed, Z. B. Zainol, B. Zaidan, and A. H. Alamoodi, “A systematic review on systems-based sensory gloves for sign language pattern recognition: An update from 2017 to 2022,” *IEEE Access*, vol. 10, pp. 123358–123377, 2022.
- [30] I. Mendez, B. W. Hansen, C. M. Grabow, E. J. Smedegaard, N. B. Skogberg, X. J. Uth, A. Bruhn, B. Geng, and E. N. Kamavuako, “Evaluation of the myo armband for the classification of hand motions,” in *2017 International Conference on Rehabilitation Robotics (ICORR)*, pp. 1211–1214, IEEE, 2017.
- [31] P. Visconti, F. Gaetani, G. A. Zappatore, and P. Primiceri, “Technical features and functionalities of myo armband: An overview on related literature and advanced applications of myoelectric arm-bands mainly focused on arm prostheses,” *International Journal*

- on Smart Sensing and Intelligent Systems*, vol. 11, no. 1, p. 1, 2018.
- [32] A. Tyagi and S. Bansal, “Hand anatomy and neural network-based recognition for sign language,” *IETE Journal of Research*, vol. 70, no. 2, pp. 1572–1584, 2024.
- [33] H. Bourekouche, “Tinymml for low-power embedded intelligent systems: A review,” *Available at SSRN 5431335*.
- [34] P. Warden and D. Situnayake, *Tinymml: Machine learning with tensorflow lite on arduino and ultra-low-power microcontrollers*. O’Reilly Media, 2019.
- [35] S. Han, J. Pool, J. Tran, and W. Dally, “Learning both weights and connections for efficient neural network,” *Advances in neural information processing systems*, vol. 28, 2015.
- [36] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2704–2713, 2018.
- [37] S. Zhuo, H. Chen, R. K. Ramakrishnan, T. Chen, C. Feng, Y. Lin, P. Zhang, and L. Shen, “An empirical study of low precision quantization for tinymml,” *arXiv preprint arXiv:2203.05492*, 2022.

- [38] H. A. Abushahla, D. Varam, A. J. Panopio, and M. I. AlHajri, “Quantized neural networks for microcontrollers: A comprehensive review of methods, platforms, and applications,” *arXiv preprint arXiv:2508.15008*, 2025.
- [39] J. Zhang, Y. Zhou, and R. Saab, “Post-training quantization for neural networks with provable guarantees,” *SIAM journal on mathematics of data science*, vol. 5, no. 2, pp. 373–399, 2023.
- [40] T. Liang, J. Glossner, L. Wang, S. Shi, and X. Zhang, “Pruning and quantization for deep neural network acceleration: A survey,” *Neurocomputing*, vol. 461, pp. 370–403, 2021.
- [41] D. Blalock, J. J. Gonzalez Ortiz, J. Frankle, and J. Gutttag, “What is the state of neural network pruning?,” *Proceedings of machine learning and systems*, vol. 2, pp. 129–146, 2020.
- [42] F. Martinini, A. Enttsel, A. Marchioni, M. Mangia, R. Rovatti, and G. Setti, “Structured pruning in deep neural networks with trainable probability masks,” in *2023 IEEE 66th International Midwest Symposium on Circuits and Systems (MWSCAS)*, pp. 1020–1024, IEEE, 2023.
- [43] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” *arXiv preprint arXiv:1503.02531*, 2015.

- [44] Q. Zhu, H. Zhuang, M. Zhao, S. Xu, and R. Meng, “A study on expression recognition based on improved mobilenetv2 network,” *Scientific Reports*, vol. 14, no. 1, p. 8121, 2024.
- [45] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, *et al.*, “Searching for mobilenetv3,” in *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 1314–1324, 2019.
- [46] STMicroelectronics, “Stm32n6 series overview – intelligent microcontrollers with integrated neural processing unit,” datasheet, STMicroelectronics.
- [47] Arm Ltd., “Cortex-m55 processor technical reference manual,” tech. rep., Arm Ltd., Cambridge.
- [48] STMicroelectronics, “An5951: Stm32n6 architecture and memory subsystem design,” Application Note AN5951, STMicroelectronics, 2024.
- [49] STMicroelectronics, *STM32Cube.AI Developer Guide*. STMicroelectronics, version 8.0 ed.
- [50] xandrew94x, “LIS-Detection - github repository.” GitHub Repository.